

Proving the Program You Run

Mutable borrows inside a dependent type theory, by observation and closing off

Abstract. Verified systems code today is written twice: once as the efficient imperative program that runs, and once as a pure model that the proofs are about, joined by a refinement proof or a translation. We present Ochr, a core calculus in which the two are the same text. Ochr combines Rust-style mutable borrows with a Lean-style dependent type theory, and its definitional equality unfolds in-place code as Lean’s unfolds pure code. Three ideas make this work. First, an *observational* equality Id A t u compares two computations by what they return and what they leave behind in the places they may write; it is not a new primitive but computes to ordinary equality between observations. Second, a call whose body is stuck on an unknown value is *closed off*: its result, and the final content of every place it borrowed, become *sealed programs*, closed source programs that own their state. Sealed programs are the backward functions of Aeneas, but written in the source language, so the programmer never sees a second language. Third, induction hypotheses are typed in the caller’s environment, where the borrow structure around the recursive argument performs the congruence step that a pure proof would write by hand. Equations between an in-place function and another computation are then proved by plain structural recursion. We give the calculus with a complete formal definition; worked examples on numbers, lists and binary search trees, including a function that returns a borrow into its argument; an executable Lean implementation of the checker, whose test suite contains every false proof found while designing the calculus and records which rule rules out each; and a mechanised proof, for a first-order fragment, that a call affects only what it is passed and that sealed programs compute Aeneas’s backward functions. Consistency is a conjecture: we sketch the set-theoretic model, in the style of Lean’s, in which we expect to prove it.

1 Introduction

Here is an in-place addition on Peano numerals. It walks down the successors of $*x$ until it reaches the final Z and moves y into that spot:

```
AddM(x : &Nat, y : Nat) : Unit by x :=  
  match *x { Z => *x := y, S p => AddM(&p, y) }
```

$\&\text{Nat}$ is a mutable borrow in the sense of Rust: a unique, temporary right to read and write a Nat owned by someone else. In the successor case the pattern variable p names the predecessor field in place, and $\&p$ reborrows it for the recursive call. Here is a theorem about `AddM`, stated and proved in the same language:

```
AddMZero(x : &Nat) : Id Unit (AddM(x, 0)) () by x :=  
  match *x { Z => refl, S p => AddMZero(&p) }
```

The statement says that running `AddM(x, 0)` is indistinguishable from doing nothing: both return `()`, and both leave the number behind x as it was. The proof is structural recursion and nothing else. There is no pure model of `AddM`, no refinement relation, no loop invariant, no separation-logic assertion and no translation into another language. The checker accepts the recursive call as a proof of the successor case by running both sides of the equation on a symbolic input and comparing what they leave behind.

This paper is about the type theory that makes that possible.

1.1 The two-language problem

Verified software is usually built in one of two ways. In the first, everything is written in a pure proof assistant such as Lean or Coq, and performance comes from the compiler’s ability to turn functional updates into in-place

ones where it can prove the old value is dead. In the second, the efficient program is written in an imperative language, and a separate pure specification is related to it by a refinement proof: Verus [18] and Creusot [11] check Rust against specifications discharged by SMT, VeriFast checks C against separation-logic contracts, and Aeneas [15] translates safe Rust into a pure Lean program about which the user then proves theorems.

Both routes keep two artifacts. In the second route this is visible: there is a program and there is a specification, and the proof is about their relationship. In the first route it is hidden but present: the program the proof is about is not the program that runs, and the gap is bridged by an optimisation the programmer cannot state or rely on.

Aeneas comes closest to closing the gap. Its insight is that Rust’s ownership discipline makes mutation equivalent to state passing: a function taking $x : \&\text{mut } T$ is a pure function returning, besides its result, a *backward function* that computes the final value of $*x$. Aeneas performs this translation once, eagerly, for a whole program, and the user proves theorems about the translated Lean code, where the backward functions appear as ordinary definitions. The user therefore works in two languages at once, and must understand how one maps onto the other. We show that this translation can be performed **inside the type checker, lazily, as definitional equality, and in the source language itself.**

1.2 Three ideas

Observational equality. The type $\text{Id } A \ t \ u$ compares two computations t and u of result type A , run from the current state. It holds when they return equal values and leave equal contents in every place they may write. Id is not a new primitive with its own introduction and elimination rules. At a given environment it computes to an ordinary propositional equality between two *observations*, the tuples of result and final contents, and that equality computes further by the structure of the data in the manner of observational type theory [5]. The places a computation may write are read off its syntax, so they are an output of the computation like its result. This is the sense in which Ochr has no pure/impure divide: any program may appear in a type, and what it does to its environment is part of what the type talks about.

Closing off. Definitional equality in Ochr is normalisation by a deterministic symbolic machine: two terms are equal when the machine sends them to the same normal form. On symbolic inputs the machine gets stuck, and a dependent type theory needs a name, a *neutral term*, for every stuck computation. A stuck pure computation names itself: Lean’s $\text{Nat.add } n \ 0$ is its own normal form. A stuck effectful call does not, because part of what it produces lives in the places it borrowed. We *close off* such a call: its result and the final content of each borrowed place are replaced by *sealed programs*, closed source programs that own their state. For AddM on an unknown σ , the borrowed place receives $[\text{let } c = \sigma; \text{AddM}(\&c, \ 0); \ c]$, which is the pure wrapper a programmer would write anyway. When a later case split refines σ , the sealed program runs again. Sealed programs are exactly Aeneas’s backward functions, but they are written in the source language and generated on demand, so the programmer never writes or reads a second language. A function that returns a borrow is closed off into a sealed program with a hole awaiting the final value of the returned borrow; this is Aeneas’s region abstraction, in the same form.

The environment does the congruence. When the checker types the recursive call $\text{AddMZero}(\&p)$, it evaluates the callee’s statement in the caller’s environment, where $\&p$ points into the predecessor field of the caller’s number. The observation therefore sees the successor that surrounds the recursive call, and the induction hypothesis arrives already wrapped in S : it is literally the goal. The reasoning that a proof about a state-passing translation must do by hand, relating the callee’s statement about *its* borrowed place to the caller’s statement about the caller’s own data, has been performed by evaluation; for natural numbers it amounts to the step $\text{cong } S \ ih$, and in general it is a frame argument. Soundness rests on the frame property that Rust’s ownership discipline provides: a call can affect only what it is passed.

1.3 Why this is not impossible

Pédrot and Tabareau’s fire triangle [29] shows that a type theory with substitution of arbitrary terms, dependent elimination and observable effects is inconsistent. Their notion of an observable effect concerns closed terms: a closed boolean that is observationally equivalent to neither **true** nor **false**. Ownership rules this out: a closed

Ochr program can mutate only places it created itself, so it behaves like the value it computes. The danger reappears for *open* terms, whose effects on the environment are observable. Ochr answers it as their call-by-value analysis does, by keeping dependent elimination and never substituting an arbitrary term into a type: a type contains only values, and a computation mentioned in a type contributes only its observation, run on a private copy of the environment. They show that the call-by-value type theory their translation yields has no dependent **let** and no large elimination; its types must be syntactic values. Ochr has both, because its typing judgement runs the term and substitutes the value it computes, which on symbolic inputs may be a neutral. The price is a disagreement of a new kind. A statement is evaluated once through the stuck-computation machinery, at a definition’s generic call, and again directly, at every instance, and any step on which the two paths disagree is a proof of false; we found several such steps while designing the calculus (Section 6). This resembles what Pédrot and Tabareau, discussing call-by-name, call a desynchronisation between the effects performed in a term and in its type, but the mechanism differs. Ochr keeps its two paths synchronised by making every decision they must agree on (which terms are erased, what a stuck block captures, which loans a match ends) from syntax rather than from normal forms.

1.4 Contributions

- A core calculus, Ochr, combining mutable borrows with a dependent type theory with inductive definitions and a universe of proof-irrelevant propositions, whose logical connectives are themselves inductive definitions and whose definitional equality unfolds imperative code (Section 3, Section 4).
- *Closing off*, which gives every stuck effectful call a neutral form made of sealed source programs, including calls that return borrows (Section 4).
- An observational equality between computations that is derived rather than primitive (Section 5).
- Worked examples, among them the equivalence of **AddM** with a version that first obtains a borrow of the final node and then writes through it, each proved by bare structural recursion (Section 2).
- The properties Ochr is designed to have, each with its evidence (Section 7): a mechanised proof, for a first-order fragment, that the machine preserves well-formedness, that a call affects only what it is passed, and that sealed programs compute the backward functions of Aeneas; and, as conjectures with sketches, consistency in a set-theoretic model in the style of Carneiro’s model of Lean, where the backward functions reappear, and the agreement of the checker’s two evaluation paths.
- An executable implementation of the checker in Lean 4 that checks every example in the paper, with a regression test for every false proof found while designing the calculus and a ledger of which rule each depends on (Section 8).

2 Ochr by example

This section introduces Ochr through its running examples. Everything is informal here; Section 3 onwards makes it precise.

2.1 Imperative code is evaluated inside types

Ochr programs are made of definitions with parameters and a body. Borrow types $\&T$ are Rust’s mutable references; places are variables x , dereferences $*p$ and the predecessor field $p.1$ of a number. A **match** on a place binds its pattern variable to a *sub-place*, not a copy, so in **match** $*x$ { $Z \Rightarrow \dots$, $S\ p \Rightarrow \dots$ } the name p stands for the field $(*x).1$ and $\&p$ reborrows it.

Add wraps **AddM** behind a pure interface. It owns its first argument, lends it to **AddM**, and returns it:

```
Add(x : Nat, y : Nat) : Nat := AddM(&x, y); x
```

Because definitional equality is evaluation, **Id Nat (Add(2, 3)) 5** holds by **refl**: the checker runs **Add(2, 3)**, which runs **AddM** in place on a local copy, and compares the result with 5. There is nothing special about imperative code here, and nothing special about **Add** being “pure”: it is simply a program whose observable output is its result.

The machine that does this is the symbolic semantics of Aeneas’s low-level borrow calculus [15]. An environment Ω maps variables to values; borrowing x moves its content into the borrow and leaves a *loan* behind; ending the borrow moves the content back. Running `Add(2, 3)` passes through these states:

```
{ x ↦ 2 }                                // enter Add
{ x ↦ loan0 } ⊢ AddM(borrow0 2, 3)      // &x: the content moves into the borrow
{ x ↦ loan0 | x' ↦ borrow0 (S loan1), ... } // AddM matched S and reborrowed the field
...
{ x ↦ 5 }                                // borrows ended, contents returned
```

2.2 Symbolic inputs, and what a stuck call leaves behind

To reason about all numbers at once, the checker runs programs on *abstract values* σ , which play the role of Lean’s free variables. Running `Add( $\sigma$ , 0)` borrows $x \mapsto \sigma$ and calls `AddM(borrow0  $\sigma$ , 0)`, whose body immediately gets stuck: it must match on σ .

In a pure type theory a stuck term is its own normal form. Here the call has also produced something that lives in the environment, namely the final content of the place it borrowed, and that content needs a name. We *close off* the call: the call returns, and the loan it held is filled with a *sealed program*, a closed source program that owns its state and computes that content:

```
{ x ↦ loan0 } ⊢ AddM(borrow0  $\sigma$ , 0)
→ { x ↦ [let c =  $\sigma$ ; AddM(&c, 0); c] } ⊢ ()
```

Write $N(\sigma)$ for `[let c =  $\sigma$ ; AddM(&c, 0); c]`. It is a neutral value, like a stuck `Nat.rec` in Lean, and it is literally the body of `Add( $\sigma$ , 0)`: the sealed program *is* the pure wrapper, generated on demand. It computes what Aeneas’s backward function for `AddM` computes [15], without leaving the source language (Section 4 says for which fragment). When a later case split refines σ to $S \sigma'$, $N(S \sigma')$ runs again, now makes progress through one successor, closes off the inner recursive call, and normalises to $S N(\sigma')$.

2.3 Equality between computations

The type `Id Unit (AddM(x, 0)) ()` compares two computations. It is evaluated at the environment where it is formed. Each side runs on its own copy of that environment; the result and the final contents of the places the side may write form its *observation*; and `Id` computes to ordinary equality between the two observations. For a definition taking $x : \&\text{Nat}$, the checker works at the definition’s *generic call* `AddMZero(&c)` from `{ c ↦  $\sigma$  }`, so the place that may be written is `c`, the owner of x ’s loan:

```
[[AddM(x, 0)]] = ((), N( $\sigma$ ))           // the call is stuck, so c receives the sealed program
[[()]]         = ((),  $\sigma$ )             // nothing happens; ending x's borrow returns  $\sigma$  to c

Id Unit (AddM(x, 0)) () ≡ Eq Unit () () ∧ Eq Nat N( $\sigma$ )  $\sigma$  ≡ Eq Nat N( $\sigma$ )  $\sigma$ 
```

`Id` is a conjunction of equations, one between the results and one for each place written; a reflexive equation is τ , and τ is a unit for $\wedge$. Here τ and $\wedge$ are not primitives: they are `True` and `And`, ordinary inductive declarations in `Prop`, and `refl` is `True`’s constructor. The statement “running `AddM(x, 0)` has no effect” has become the ordinary proposition that $N(\sigma)$ equals σ .

2.4 Induction hypotheses at the call site

Now the proof:

```
AddMZero(x : &Nat) : Id Unit (AddM(x, 0)) () by x :=
  match *x { Z => refl, S p => AddMZero(&p) }
```

The match on $*x$ splits on σ . In the Z branch the goal is $\text{Eq Nat } N(Z) \ Z; N(Z)$ runs to Z , the equation is reflexive and so τ , and refl proves it. In the S branch the goal is $\text{Eq Nat } N(S \ \sigma') \ (S \ \sigma')$, which normalises to $\text{Eq Nat } (S \ N(\sigma')) \ (S \ \sigma')$, and then, since Eq takes apart two successors, to $\text{Eq Nat } N(\sigma') \ \sigma'$.

The recursive call is checked by the same rule that produced the goal: the callee's statement is evaluated at the call site, with its parameter bound to the argument. At the call site the environment is

```
{ c ↦ loan0 | x ↦ borrow0 (S loan1) } and the argument is borrow1 σ'
```

because matching $S \ p$ through the borrow and taking $\&p$ leaves the successor in place, around a loan for the predecessor. The owner of the argument's loan is found by following it outwards: loan_1 sits inside x 's borrow, whose loan sits in c . So the recursive call's statement observes c , and running AddM on the argument fills loan_1 with $N(\sigma')$, after which c holds $S \ N(\sigma')$:

```
[[AddM(x', 0)]] at the call site = ((), S N(σ'))
[[()]] at the call site = ((), S σ')
AddMZero(&p) : Eq Nat (S N(σ')) (S σ') ≡ Eq Nat N(σ') σ'
```

This is the goal. The callee's statement is about the place it borrowed; evaluated at the call site, it is about the caller's number, successor included. Relating the two is the frame argument that a proof about a translated program makes by hand, and here it was done by evaluation. The recursive call is a proof, and proofs are erased at runtime, so the checker does not run it: its borrow argument is returned unchanged.

The same theorem about the pure wrapper, $\text{AddZero}(x : \text{Nat}) : \text{Id Nat } (\text{Add}(x, 0)) \ x$, has the same goal, since $\text{Add}(x, 0)$ writes nothing outside itself. It is proved by lending the predecessor field of the owned x to the in-place lemma, or by recursion on a copy of it; indeed $\text{AddZero}(x) := \text{AddMZero}(\&x)$ type-checks, because both statements normalise to the same proposition. Neither proof mentions a model of AddM : the statement is about the program itself.

2.5 Returning a borrow

A more idiomatic in-place addition first finds the final node and then writes through it:

```
TailM(x : &Nat) : &Nat by x := match *x { Z => x, S p => TailM(&p) }
AddM'(x : &Nat, y : Nat) : Unit := let t = TailM(x); *t := y
```

TailM returns a borrow into its argument. When it is stuck on σ , the place it borrowed cannot be given its final content yet, because that depends on what the caller will later write through the returned borrow. Closing off such a call returns a borrow of a fresh k and fills the argument's loan with a sealed program that has a *hole* loan_k for the returned borrow's final content:

```
{ c ↦ loan0 } ⊢ TailM(borrow0 σ)
→ { c ↦ [let c = σ; let r = TailM(&c); *r := loank; c] }
⊢ borrowk [let c = σ; let r = TailM(&c); *r]
```

In AddM' , the caller writes y through t and drops it; ending borrow_k substitutes the final content y for the hole. What remains in c is $[let \ c = \ \sigma; \ let \ r = \ \text{TailM}(\&c); \ *r := \ y; \ c]$: the effect of AddM' , as a program. The hole plays the role of Aeneas's region abstraction for this one call: it records only the *final* value written through the returned borrow.

The equivalence of the two additions is proved by the same bare recursion:

```
AddMEq(x : &Nat, y : Nat) : Id Unit (AddM(x, y)) (AddM'(x, y)) by x :=
match *x { Z => refl, S p => AddMEq(&p, y) }
```

In the successor branch, re-running AddM' 's sealed program on $S \ \sigma'$ unfolds TailM once, closes off its inner call with a fresh hole, and the pending write $*r := y$ then fills that hole; on the other side the recursive call's statement, evaluated at the call site, performs the same steps through the caller's borrow. Both sides arrive

at $\text{Eq Nat } (S A(\sigma', y)) (S B(\sigma', y))$, where A and B are the sealed programs for the two additions on the predecessor. The theorem for an owned number follows by lending it: $\text{AddMEqOwned}(x : \text{Nat}) : \text{Id Unit } (\text{AddM}(\&x, 0)) (\text{AddM}'(\&x, 0)) := \text{AddMEq}(\&x, 0)$.

2.6 Proofs about the current state

Dependent types are not only for stating theorems after the fact. A function can demand, as an argument, a proof about the *current* contents of a borrow. Here is in-place subtraction, which is total only when the subtrahend is no larger than the number:

```
Le(a : Nat, b : Nat) : Prop by a :=
  match a { Z => True, S a' => match b { Z => False, S b' => Le(a', b') } }

SubM(x : &Nat, y : Nat, h : Le(y, *x)) : Unit by y :=
  match y { Z => (), S q => match *x { Z => match h {}, S p => *x := p; SubM(x, q, h) } }
```

The type of h mentions $*x$, the content of the borrow at the moment of the call. Where $*x$ is Z but y is not, h has type $\text{Le}(S q, Z)$, which computes to False : the case cannot arise, and the match with no arms, $\text{match } h \{ \}$, says so. Inside SubM , after $*x := p$ has overwritten that content, the recursive call needs a proof of $\text{Le}(q, *x)$ about the *new* content; the old hypothesis h , of type $\text{Le}(S q, S p)$, provides it, because that type was formed when h was bound and normalises to $\text{Le}(q, p)$.

Now a caller that first adds and then subtracts:

```
LeAdd(n : Nat, m : Nat) : Le(n, Add(n, m)) by n := match n { Z => refl, S n' => LeAdd(n', m) }

AddSub(x : &Nat, y : Nat) : Unit := let old = *x; AddM(&*x, y); SubM(x, old, LeAdd(old, y))
```

At the call to SubM , $*x$ has just been mutated in place by AddM , so on symbolic input it holds the sealed program $N(\sigma, y) = [\text{let } c = \sigma; \text{AddM}(\&c, y); c]$, and SubM demands a proof of $\text{Le}(\sigma, N(\sigma, y))$. The lemma LeAdd is about the *pure* Add of the snapshot old , and its type is $\text{Le}(\sigma, \text{Add}(\sigma, y))$. The two meet because $\text{Add}(\sigma, y)$ normalises to the very same sealed program: the in-place computation and the pure one are the same program to the type checker. A proof about the pure function is accepted where a proof about the mutated state is required, with no bridging lemma. Using the snapshot after further mutation, $\dots; \text{AddM}(\&*x, y); *x := Z; \text{SubM}(x, \text{old}, \text{LeAdd}(\text{old}, y))$, is rejected, since the requirement then mentions Z .

2.7 Trees

Nothing above is specific to numbers. With several constructors and several fields, a pattern variable names a field place, and the environment keeps every field that a recursive call does not touch. In-place insertion into a binary search tree recurses into the left or the right subtree according to a comparison, so which place is mutated depends on a value. Its pure version is not written separately; like Add , it runs the in-place one on a copy:

```
InsertM(t : &Tree, k : Nat) : Unit by t :=
  match *t { Leaf          => *t := Node(Leaf, k, Leaf),
            Node(l, v, r) => let b = Lt(k, v);
                           match b { true => InsertM(&l, k), false => InsertM(&r, k) } }
Insert(t : Tree, k : Nat) : Tree := InsertM(&t, k); t
```

A property of the in-place code is then stated and proved directly: insertion adds one node.

```
SizeInsert(t : Tree, k : Nat) : Id Nat (S (Size(t))) (Size(Insert(t, k))) by t :=
  match t { Leaf => refl,
            Node(l, v, r) => let b = Lt(k, v); match b {
              true => J(Nat, S (Size(l)), Size(Insert(l, k)),
                λ(z : Nat) : Prop => Id Nat (S (S (Add(Size(l), Size(r))))) (S
```

```

(Add(z, Size(r))),
      SizeInsert(l, k), refl),
  false => J(Nat, Add(Size(l), S (Size(r))), S (Add(Size(l), Size(r))),
    λ(z : Nat) : Prop => Id Nat (S z) (S (Add(Size(l), SizeInsert(r,
k))))) ,
      AddS(Size(l), Size(r)),
      J(Nat, S (Size(r)), SizeInsert(r, k)),
      λ(z : Nat) : Prop => Id Nat (S (Add(Size(l), S (Size(r))))) (S
(Add(Size(l), z))),
      SizeInsert(r, k), refl)) } }

```

The comparison $\text{Lt}(\sigma_k, \sigma_v)$ is stuck, so the split on b is a split on a sealed program: the checker names its value by a fresh abstract value, and replaces every derivation of the same closed program by it, including those produced later when the goal's sealed programs run again. In each arm, `Insert`'s sealed program runs one step and leaves the other subtree and the key in place, carried by the environment. The proof is not bare recursion: without rewriting tactics, each arm rewrites with the induction hypothesis by `J` with a hand-written motive, and the `false` arm needs an arithmetic lemma, `AddS` : $x + S y = S (x + y)$, proved in place by bare recursion and transferred to `Add` by lending, as `AddZero` was. This is the honest cost of a property that is not an equation between two recursions of the same shape.

2.8 Branching

A `match` on an abstract value in the middle of a function is closed off like a call: each arm is checked separately, and the rest of the function once. This covers borrows whose origin depends on the branch, as in `AddToOne(b : Nat, x1 : &Nat, x2 : &Nat, y : Nat) : Unit := let r = match b { Z => x1, S _ => x2 }; AddM(r, y)`, whose closed-off match returns a borrow with a hole in the sealed programs of both x_1 's and x_2 's owners.

3 The calculus

Ochr is a dependent type theory in the style of Lean's kernel [8], extended with places, mutable borrows and sequencing. This section gives its syntax and the runtime structures of its evaluator. Data and the logical connectives are introduced alike, by inductive declarations; the rules in the body are illustrated on natural numbers, and Section A gives the general definition.

3.1 Syntax

Figure 1 gives the syntax. Types and terms share one grammar, as in any pure type system. Functions are n-ary and calls are saturated: a partial application would be a closure capturing its arguments, and a closure capturing a borrow is outside the core (Section 4.3). A recursive function names the parameter it recurses on.

Inductive definitions. Every type of data and every logical connective is an inductive declaration: a name, uniform parameters, a sort (`Typeo` for data, `Prop` for propositions) and any number of constructors with named fields. Natural numbers, the unit type and pairs are declarations (the checker builds `Nat` and `Unit` in, with the same behaviour), and so are `False`, the proposition with no constructors, `True`, with one constructor and no fields, and `And(P, Q)`, with one constructor whose two fields are proofs of P and of Q . τ , $P \wedge Q$, $\langle h, k \rangle$ and `refl` are notation for them. Their eliminations are matches: `ex falso` is the match with no arms, `match h { }`, and a proof of $P \wedge Q$ is taken apart by `match h { Intro(l, r) => ... }`. There is one mechanism for all of them, and it is the one Lean, Coq and Agda use. Constructors also take their declaration's parameters as leading arguments, `Intro(P, Q; h, k)`, which we omit when they can be inferred. Fields are first-order: they may mention the type being declared, other declared types and the parameters, but not Π or $\&$, which is strict positivity in its simplest form.

`Eq` is the one primitive proposition. It is not an inductive family, for two reasons: it computes by the structure of the values it compares, as in observational type theory (Section 5), and the core has no indexed families.

$t, u, A, B ::= x \mid \text{Prop} \mid \text{Type}_i$	variables, universes
$\mid \Pi(x_1 : A_1 \dots x_n : A_n).B \mid \text{fix } f(x_1 : A_1 \dots x_n : A_n) : B \text{ by } x_j := t \mid t(u_1, \dots, u_n)$	functions, calls
$\mid D(a_1, \dots, a_m) \mid C(t_1, \dots, t_k)$	inductive types, constructors
$\mid \text{Eq } A \ t \ u \mid J(A, a, b, P, h, t)$	equality
$\mid \&A \mid p \mid \&p \mid p := t \mid \text{let } x = t; u \mid \text{let } x : A = t; u \mid t; u$	places and borrows
$\mid \text{match } p \{ C_1(\bar{y}_1) \Rightarrow t_1, \dots, C_n(\bar{y}_n) \Rightarrow t_n \} \quad (n \geq 0)$	case analysis
$\mid \text{Id } A \ t \ u$	computation equality
$p ::= x \mid *p \mid p.g$	places (g a field)
$d ::= \text{inductive } D \ (x_1 : A_1 \dots x_m : A_m) : s := C_1(\overline{g_1 : B_1}) \mid \dots \mid C_n(\overline{g_n : B_n})$	declarations
$\text{inductive Nat} : \text{Type}_0 := \mathbb{Z} \mid S(\text{pred} : \text{Nat})$	
$\text{inductive Unit} : \text{Type}_0 := \text{Tt}$	
$\text{inductive False} : \text{Prop}$	
$\text{inductive True} : \text{Prop} := \text{I}$	
$\text{inductive Pair } (A : \text{Type}_0)(B : \text{Type}_0) : \text{Type}_0 := \text{Mk}(\text{fst} : A, \text{snd} : B)$	
$\text{inductive And } (P : \text{Prop})(Q : \text{Prop}) : \text{Prop} := \text{Intro}(l : P, r : Q)$	

Figure 1: Syntax of Ochr, and the library declarations. A declaration has sort $s \in \{\text{Type}_0, \text{Prop}\}$ and $n \geq 0$ constructors. $S \ t$ abbreviates $S(t)$; $()$, $A \times B$, (a, b) , $\top$, **refl**, $P \wedge Q$ and $\langle h, k \rangle$ are notation for **Tt**, **Pair**(A, B), **Mk**(a, b), **True**, **I**, **And**(P, Q) and **Intro**(h, k), and $p.1$, $p.2$ name a place's first and second fields. **Eq** is the one primitive proposition. A **fix** without **by** is an ordinary λ .

The imperative fragment is small. A *place* p is a variable, a dereference $*p$, or a field $p.g$ of a constructor value, such as the predecessor field $p.1$ of a number. A place used as a term reads it; $\&p$ borrows it; $p := t$ assigns it; **let** $x = t; u$ introduces a new place x . The pattern variables of a **match** are *sub-places*: in **match** $p \{ S \ y \Rightarrow u \}$, y stands for $p.1$, and nothing is copied.

$\&A$ is the type of a mutable borrow of an A , and A must be data: an inductive type in Type_0 , never a universe, a Π -type or a proposition. Whether A is data is read from its evaluated head, so a type variable is not known to be data and there is no generic $\&A$ for an arbitrary $A : \text{Type}_0$ (Section 10). $\&$ occurs only at the top of a type as written, as the type of a variable, parameter or result; never inside another type, and never as the result of computing one. So there are no borrows stored in data structures and no borrows of borrows. There are no shared borrows, no loops and no 'static borrows: a borrow a function returns derives from one of its borrow arguments (Section 10). Recursion is structural.

The propositional fragment is Lean's: **Prop** is an impredicative universe with definitional proof irrelevance, erased at runtime, and **Eq** is its equality. **Id** $A \ t \ u$ compares two *computations* t and u of type A ; it is a derived proposition whose computation rule is observation (Section 5).

3.2 Values and environments

$v, w ::= C(v_1, \dots, v_k) \mid \star \mid f \mid \langle \bar{\kappa} \vdash \text{fix } f(\bar{x} : \bar{A}) : B \dots := t \rangle \mid \text{types}$	data, proofs, functions, closures, types
$\mid \text{borrow}_\ell v \mid \text{loan}_\ell \mid \perp$	borrows, loans, moved-out
$\mid n$	neutrals
$n ::= \sigma \mid [t]$	abstract values, sealed programs
$\Omega ::= \cdot \mid \Omega, x : A \mapsto v \mid \Omega \mid \Omega'$	bindings, grouped in frames

Figure 2: Runtime structures. A closure is code with its captured values $\bar{\kappa}$, and a Π -type is one too, $\langle \bar{\kappa} \vdash \Pi(\bar{x} : \bar{A}).B \rangle$ (Section 4.3).

The evaluator manipulates the values of Figure 2. Following the low-level borrow calculus of Aeneas [15], a borrow carries the borrowed content with it: **borrow** $_\ell \ v$ is a borrow, identified by ℓ , whose current content is v , and **loan** $_\ell$ is the placeholder left at the place it was taken from. When the borrow ends, its content is put back where the loan is. We depart from Aeneas in one respect: **a loan is a variable bound by its borrow**. It may occur anywhere inside a value, including inside a sealed program, and inside sealed programs it may occur more than once; ending the borrow substitutes its content for every occurrence. $\perp$ marks a place whose content has been moved out, and $\star$ is the value of a proof.

Neutral values are stuck computations. An *abstract value* σ is an unknown value of a known type, playing the role of a free variable in Lean’s kernel: the checker introduces one for each parameter of a definition it checks, and for each field that a case split or a generalisation exposes; concrete evaluation never introduces one. A *sealed program* $[t]$ is a closed program whose run is stuck on an abstract value. Sealed programs are produced by closing off stuck calls (Section 4) and are the only neutral form besides abstract values. A value is *borrow-free* if it contains no borrow and no live loan.

An environment Ω is a stack of frames, one per active call, each binding variables to values. `content( $\Omega$ ,  $p$ )` follows a place through variables, borrows (`* $p$` looks inside `borrow_ℓ v` at v) and fields (`$p.1$` looks inside `S v` at v). We write $\Omega[p \mapsto v]$ for the environment with the content of p replaced.

4 Evaluation and closing off

Definitional equality in Ochr is normalisation by one deterministic machine. With effects, rewriting-anywhere is not confluent: evaluating $(\lambda y. (x := 2; y))(x := 1)$ argument-first and substituting-first leave x holding different numbers. A fixed call-by-value order restores uniqueness, and the machine then plays the role that normalisation by evaluation plays for pure type theories [2]: two terms are definitionally equal when the machine sends them to the same normal form. On concrete inputs it is an ordinary interpreter. On abstract inputs it runs until it has to inspect an abstract value, and the question the rest of this section answers is what it returns then. The judgement is big-step: $\langle \Omega, t \rangle \Downarrow \langle \Omega', v \rangle$ runs t from environment Ω to value v and environment Ω' , and a run may instead be *stuck* (it needs to inspect a neutral) or fail with a borrow error.

4.1 Borrows

$$\begin{array}{c}
\frac{\text{borrow}_\ell v \text{ in } \Omega}{\Omega \rightsquigarrow \Omega[\text{borrow}_\ell v := \perp][v/\text{loan}_\ell]} \text{ END} \quad \frac{v = \text{content}(\Omega, p)}{\langle \Omega, \&p \rangle \Downarrow \langle \Omega[p \mapsto \text{loan}_\ell], \text{borrow}_\ell v \rangle} \text{ BORROW} \\
\\
\frac{v = \text{content}(\Omega, p) \text{ borrow-free}}{\langle \Omega, p \rangle \Downarrow \langle \Omega, v \rangle} \text{ READ} \\
\\
\frac{\text{content}(\Omega, p) = \text{borrow}_\ell v}{\langle \Omega, p \rangle \Downarrow \langle \Omega[p \mapsto \perp], \text{borrow}_\ell v \rangle} \text{ MOVE} \quad \frac{\langle \Omega, t \rangle \Downarrow \langle \Omega_1, v \rangle \quad \Omega_2 = \text{drop}(\Omega_1, \text{content}(\Omega_1, p))}{\langle \Omega, p := t \rangle \Downarrow \langle \Omega_2[p \mapsto v], () \rangle} \text{ ASSIGN}
\end{array}$$

Figure 3: Borrows. Every rule that accesses a place p is preceded by [Access]: end every borrow whose loan lies on the path to p or inside its content (for a match, which inspects only the head: on the path, at the head, or anywhere inside a neutral head). **drop** ends the borrows inside a dropped value and fails on a live loan in a dropped owned value.

Figure 3 gives the rules for borrows. They are Aeneas’s, simplified by treating loans as variables.

Ending a borrow [End] replaces it by $\perp$ and substitutes its content for its loan. There is no side condition: if the content itself contains loans (because something has reborrowed part of it), they travel with it, and the reborrows remain valid. Ending is allowed at any time; the machine does it lazily, when a place is accessed. [Access] makes every access exclusive: before a place is read, borrowed, assigned or matched on, every borrow whose loan lies on the path to it or inside its content is ended. A borrower that has been ended holds $\perp$, and any later use of it is an error. This is the whole of the borrow checker: exclusivity of mutable borrows is not a separate analysis but a consequence of running the program.

Reading a place copies its content if the content is borrow-free [Read] and moves it out otherwise [Move]. Data is therefore copied, and borrows, which must stay unique, are moved; reborrowing is explicit (`&* $x$`). A compiled program should implement a last-use copy as a move, so that **AddM** performs no allocation; an affine usage discipline outside the core would license this, but we have not defined it, nor proved that the compiled program agrees with the machine (Section 10). Inside types copying is essential: `Id Nat (Add( $x$ ,  $x$ )) x` must at least be a well-formed statement, and so must `$x + x = 2 \cdot x$` .

4.2 Calls and matches

A call evaluates its arguments left to right, each into a temporary of the current frame, so that `[Access]` can see and end them: `f(&x, &x)` ends the first borrow when the second is taken, and the call then fails reading `⊥`. It then pushes a frame binding the parameters, runs the body, and pops the frame, dropping its bindings. Popping a frame that still lends out one of its owned places is an error: something would outlive the place it borrows.

Types and proofs are erased at runtime, and the machine treats them accordingly: an erased term is evaluated on a *private copy* of the environment, argument evaluation included, and the copy is then discarded. Which terms are erased is read from syntax and declarations, never from a normal form (Section 6): a call is erased when its callee’s codomain *term* is a sort or has declared sort **Prop**, and any other term when it stands in a type position or its declared type has sort **Prop**. A proof may use mutation freely on its copy, but it cannot affect the real environment, so running a proof and skipping it are indistinguishable and the machine skips it, returning `*`. A call whose head is itself a neutral (an abstract function, or a sealed program) is stuck at once.

`match p { Z => t, S y => u }` inspects the head of `p`’s content. `Z` selects the first arm, and `S v` selects the second with `y` standing for the sub-place `p.1`. Through a borrow, `match *x` leaves the successor in the borrowed content, so `&y` then reborrows the predecessor field in place: `x ↦ borrow0 (S loan1)` and the new borrow is `borrow1 v`. If the head is a neutral, the run is stuck. A match on a proof never inspects it, since every proof is `*`: whether a match is on a proof is read from the declarations of the constructors its arms name, and such a match is decided by the proof’s type (Section 6.5).

4.3 Functions and closures

A `fix` evaluates to a *closure*: its code together with the current values of its free variables, written $\langle \bar{\kappa} \vdash \mathbf{fix} \ f(\bar{x} : \bar{A}) : B \dots := t \rangle$. The captured values $\bar{\kappa}$ are kept in the order of the bindings they were read from, each with that binding’s declared type and whether it was declared a proof. A Π -type is formed the same way, as $\langle \bar{\kappa} \vdash \Pi(\bar{x} : \bar{A}).B \rangle$. A top-level function captures nothing and is just its name, its code being in the global signature.

Capturing is reading. Each free variable is read when the closure is formed, as by `[Read]`, so a borrow that lends it out is ended first, and the closure keeps a copy. What can be captured is what can be copied: data, proofs (`*`), types, other closures, abstract values and sealed programs. A borrow cannot be captured, nor a place that has been moved out, and forming a closure that would capture one is an error. So a `λ` inside a function that takes `x : &Nat` cannot mention `x`; it copies the number first, as in `let n = *x; let f = (λ(y : Nat) : Nat => n)`. A closure may still take borrows as parameters, as `λ(z : &Nat) : Unit => AddM(z, 0)` does.

Three things follow. Captures are snapshots: later mutation of a variable does not change a closure that captured it, which is the mechanism by which a type is formed once (Section 6). There is no capture by reference and no closure that mutates its environment; Rust’s `FnMut` has no counterpart. And a closure holds no borrow, so `[Read]` copies it like data; it cannot be a field of an inductive value, whose fields are first-order, nor be borrowed, since `&A` is only for data. A borrow inside a closure would be a borrow stored inside a value, which is the restriction on borrows in data (Section 10).

Calling a closure pushes one frame, holding its captured values, the closure itself if it is recursive, and the parameters, and runs `t` there. Two function values are convertible when their Π -types are, their captured values are pairwise, and their generic calls have the same observation (Section A.3). The only capture of places, in the style of Rust’s closures, is that of a closed-off stuck match (Section 4.5): its places become the arguments of a call, moved, borrowed or copied, never the contents of a stored value.

4.4 Closing off

When the body of a call is stuck, the call itself is stuck, and so is everything waiting for it. A pure type theory would return the stuck call as a neutral term. An effectful call cannot simply be returned: part of what it produces is the final content of the places it borrowed, and the caller’s environment needs those contents before it can continue. We *close off* the call instead.

the body of $f(w_1, \dots, w_n)$ is stuck, $w_i = \text{borrow}_{\ell_i} u_i$ for $i \in I$, u_i loan-free

$$L := \text{let } c_i = u_i \quad (i \in I) \quad C := f(a_1, \dots, a_n), \quad a_i = \begin{cases} \&c_i & \text{if } i \in I \\ w_i & \text{otherwise} \end{cases}$$

result type	result	each loan_{ℓ_i} becomes
Unit	$()$	$[L; C; c_i]$
borrow-free D	$[L; C]$	$[L; C; c_i]$
$\&T$	$\text{borrow}_k [L; \text{let } r = C; *r]$	$[L; \text{let } r = C; *r := \text{loan}_k; c_i]$

Figure 4: [Close]. The partial run of the body is discarded; the borrows borrow_{ℓ_i} are consumed; k is fresh.

Figure 4 gives the rule. Take the call at the point where its arguments have been evaluated. For each borrow argument $\text{borrow}_{\ell_i} u_i$, the canonical program $L; C$ recreates the situation the call was in, but with the borrowed contents owned by fresh local places c_i : it moves each u_i into c_i and calls f on $\&c_i$. This program is closed. Running it and reading c_i afterwards computes exactly what the call would have left behind the borrow, so each loan is filled with the sealed program $[L; C; c_i]$. The call returns the sealed program $[L; C]$, or $()$ if its result type is **Unit**.

Sealed programs are the backward functions of Aeneas, written in the source language. For $\text{AddM}(\text{borrow}_\sigma \sigma, y)$, the loan receives $[\text{let } c = \sigma; \text{AddM}(\&c, y); c]$, which is the body of $\text{Add}(\sigma, y)$. Where Aeneas’s translation would emit `add_m_back` σ y , Ochr writes the program that defines it. Nothing about the call is lost: when σ is later refined, the sealed program runs again.

Returning a borrow. If f returns a borrow, the contents of the places it borrowed are not determined when the call returns: they depend on what the caller will write through the returned borrow before it ends. The rule therefore returns a borrow of a fresh k , whose content is the sealed program for the returned borrow’s initial content, and fills each loan_{ℓ_i} with a sealed program that has a *hole*: it writes loan_k through the returned borrow before reading c_i . Because loans are variables, the hole is filled by the ordinary [End] rule: when the caller drops the returned borrow with final content w , w is substituted for loan_k . This records only the *final* value written through the returned borrow, which is all that the backward function of Aeneas takes as input, and the hole is the neutral form of an Aeneas region abstraction and of a RustHorn prophecy [23]. If f has several borrow arguments, the hole occurs in each of their sealed programs, and one [End] fills them all.

The precondition that each u_i is loan-free is guaranteed by [Access], which ended those loans when the argument was read or borrowed. Without it, a live loan would be copied into a sealed program and survive the borrow that binds it.

4.5 Sealed programs and refinement

Values are kept in normal form. The normal form of a sealed program $[L; C; K]$ is obtained by running it from the empty environment, with one restriction: its *head call* C may not itself be closed off (calls made inside C ’s body may). This is the analogue of the guard on fixpoint unfolding in the calculus of inductive constructions. Without it, a sealed program would re-close into itself forever; with it, a sealed program either runs to a value or stays exactly as it is. A loan whose borrow lives outside the run (a hole whose returned borrow is still live) is *inert*: the run treats it like an abstract value. A borrow error during normalisation is a type error at the point that triggered it. When a case split has generalised a sealed program to an abstract value (Section 6), every later derivation of the same closed program normalises to that value.

A *refinement* substitutes a constructor pattern for an abstract value, $\sigma := Z$ or $\sigma := S \sigma'$, when a case split learns it (Section 6). Substitution into a sealed program is followed by normalisation, which runs the sealed program again. For $N(\sigma) = [\text{let } c = \sigma; \text{AddM}(\&c, \emptyset); c]$ and $\sigma := S \sigma'$, the head call AddM now unfolds, matches the successor, reborrows the predecessor, and makes the inner recursive call, which is stuck on σ' and is closed off; so $N(S \sigma') = S N(\sigma')$. Normal forms of this shape are what makes the induction of Section 2 go through. The key property, a conjecture of Section 7, is that refinement commutes with closing off: refining the

arguments of a closed-off call and re-normalising gives the same normal form as closing off the call on the refined arguments.

Stuck blocks. A stuck `match` that is not the body of a call is closed off in the same way, as the body of a call to an anonymous function of its free places. Pattern variables are first resolved to the sub-places they denote, and the function takes the resulting maximal places the way Rust infers closure captures: a place the match moves out of, in any arm, is moved in; a place it writes or borrows is passed by borrow; a place it only reads is copied. The function's result type is the match's type, which must agree across the arms or be annotated (`let x : A = match ...`). Only a match whose arms have been type-checked is closed off, including a match inside a type, and a stuck block is erased exactly when each of its arms is a declared proof. This covers stuck matches inside types and, during checking, a stuck match that is followed by more code (Section 6).

5 Observational equality

Id A t u states that the computations `t` and `u`, both of result type `A`, are indistinguishable from where the statement is made: run from the current environment, they return equal values and leave equal contents in every place either of them may write. This section defines that notion precisely and shows that it reduces to ordinary propositional equality.

5.1 Footprints and observations

A computation affects the world only through the places it names. The places a term may **write** are read off its syntax: they are the free places that appear under `&_` or on the left of `:=`, and the borrow variables it mentions, since a borrow grants write access to someone else's place. Each of these is traced back to the owned place that will receive the write. For a place rooted at an owned variable `x`, that is `x`. For a borrow variable holding `borrow_ℓ v`, it is the set of *owners* of `ℓ`:

$$\text{owners}(\ell) = \bigcup_{\text{occurrences of loan}_\ell} \begin{cases} \text{owners}(m) & \text{if the occurrence lies inside the content of borrow}_m \\ \{x\} & \text{if it lies inside the owned binding } x \end{cases}$$

Following loans outward through enclosing borrows is what connects a reborrow with the place it ultimately writes. In the successor case of `AddMZero`, the argument `borrow1 σ'` has its loan inside the content of `x`'s `borrow0`, whose loan is in the generic caller's `c`; so the owner of the recursive call's argument is `c`, the same place the goal observes. The owners form a set because a hole left by a returned borrow can occur in several sealed programs at once. Observing only one of them would be unsound: a call that returns a borrow into one of two arguments could then be proved to leave the other unchanged whichever it picked.

The *footprint* $\mathbf{W}(t, u)$ is the union of the owners of the places `t` or `u` may write. The places are read off the syntax, after resolving pattern variables to the sub-places they denote, so the footprint does not depend on unrelated places in the environment; their owners are found by following loans, so a later refinement can shrink an owner set when it decides where a hole points, and the component it removes is one both sides agree on.

The *observation* of `t` at Ω on footprint $\mathbf{W}$ runs `t` on a private copy of Ω , ends every borrow that remains, and reads off the result and the contents of the footprint:

$$\llbracket t \rrbracket_{\Omega}^{\mathbf{W}} = (v, \Omega'(k)_{k \in W}) \quad \text{where } \langle \Omega, t \rangle \Downarrow \langle \Omega'', v \rangle \text{ and } \Omega'' \overset{*}{\rightsquigarrow} \Omega' \text{ with no borrows left}$$

If `t` is itself stuck outside any call, it is first closed off as a stuck block, so an observation always exists. Ending every remaining borrow makes the observation independent of which borrows happened to be live when `t` finished; that the order in which they are ended does not matter is a conjecture, proved for two endings (Section 7).

5.2 Id is equality of observations

$\text{Id } A \ t \ u \equiv \text{Eq } A \ r \ r' \wedge \text{Eq } T_1 \ w_1 \ w'_1 \wedge \dots \wedge \text{Eq } T_k \ w_k \ w'_k$ where $\llbracket t \rrbracket_\Omega^W = (r, \overline{w})$, $\llbracket u \rrbracket_\Omega^W = (r', \overline{w'})$, $W = W(t, u)$

$\text{Eq } D(\overline{a}) \ C(\overline{v}) \ C(\overline{w}) \equiv \text{Eq } B_1 \ v_1 \ w_1 \wedge \dots \wedge \text{Eq } B_k \ v_k \ w_k$ $\text{Eq } D(\overline{a}) \ C(\overline{v}) \ C'(\overline{w}) \equiv \text{False}$ if $C \neq C'$

$\text{Eq } A \ a \ b \equiv \top$ if $a \equiv b$ $\top \wedge P \equiv P \equiv P \wedge \top$

Figure 5: **Id** computes to a conjunction of equations between the two observations, both run from Ω on independent copies (T_i is the type of the i -th place of W). **Eq** computes on constructor values by structure (B_j are the fields' types; no fields gives $\top$) and on reflexive instances. $\top$, $\wedge$ and **False** are the library inductives of Figure 1, and **refl** : $\top$.

Figure 5 is the whole of **Id**. Both sides run from the same environment, each on its own copy; sharing one copy and running the sides in sequence would let the first side's effects change what the second observes. The result type **A** must be borrow-free: a borrow has no meaning once the computation that produced it has finished. An observation is a tuple of the machine, not a pair value, so **A** may be any borrow-free type, a proposition or a universe included.

The equality **Eq** is Lean's, living in the proof-irrelevant universe **Prop**, and it computes on constructor values by structure, in the style of observational type theory [5, 28]. Two values built by the same constructor are equal when their fields are (injectivity), two built by different constructors are not (disjointness), a reflexive equation is $\top$, and $\top$ is a unit for $\wedge$. The rules produce the library inductives **And**, **True** and **False**, so these three are known to the conversion checker, the one point where the library declarations are special. The unit laws are conversion rules, not a rewriting of stored types: a hypothesis of type $\top \wedge P$ keeps its **And** and can still be taken apart by a match. Together the rules strip away what no side changed, so that **Id Unit (AddM(x, 0)) ()** computes to the single interesting equation:

$$\text{Eq Unit } () \ () \wedge \text{Eq Nat } N(\sigma) \ \sigma \equiv \top \wedge \text{Eq Nat } N(\sigma) \ \sigma \equiv \text{Eq Nat } N(\sigma) \ \sigma.$$

Disjointness lets an observation that differs in a constructor reach **False** directly: at a definition taking $x : \&\text{Nat}$, the statement **Id Unit (*x := 0) (*x := 1)** computes to **False**, so a hypothesis of that type is eliminated by **match e {}**. An **Id** over several places computes to a conjunction, which **match h { Intro(l, r) => ... }** takes apart. Each rule identifies two propositions with the same truth value, so all of them hold in the proof-irrelevant set model of Lean's type theory [8]. **refl** proves $\top$, and so, by conversion, every reflexive equation. Transport **J(A, a, b, P, h, t)** takes its endpoints explicitly, since a reflexive **Eq A a a** computes to $\top$ and no longer records them, and it computes to **t** only when $a \equiv b$, as Lean's **Eq.rec** does. Under a hypothesis that equates unequal values a cast stays stuck, so there is no equality reflection and no value of the wrong type.

Three consequences are worth drawing out.

Effects are outputs. The observation turns every write to a place in the footprint into an extra component of the result. This is the sense in which Ochr has no pure/impure divide: a computation's effects are part of its value as seen by **Id**, and the programmer never declares them.

Owned locals are observed. The footprint includes places the current function owns, not only the places behind its borrow parameters. This is forced. A context such as $(\Box; x)$, which reads x afterwards, distinguishes $x := 6$ from $()$; if **Id** ignored owned locals it would equate them, and transport along that equation would prove **Eq Nat 6 5**.

Unobserved moves are harmless. Reading a borrow variable moves it, so the two sides of an **Id** may consume different borrow variables. Borrow variables are never in the footprint (their owners are), and ending the remaining borrows puts every borrowed content back in its owner, so what one side consumed and the other did not makes no difference to the comparison.

6 Typing

Typing is the machine run on abstract inputs, with case splitting where the checked program itself inspects an abstract value. The judgement $\Omega \vdash t \Downarrow v : A \dashv \Omega'$ says that from Ω the term t runs to the value v of type A , leaving Ω' . It returns the value, not just the type, because later types may depend on it: after `let z = (let y = &x; *y := 2; x)`, the type `Id Nat z 2` holds by `refl` only because the checker knows that z is 2. The rules for the basic forms are the evident ones, each the corresponding machine rule annotated with types, and we omit them. Figure 6 gives the rules that are particular to Ochr.

[Call-type] At the point where the arguments of $f(\bar{a})$ have been evaluated, with $f : \Pi(\bar{x} : \bar{A}) . B$: push a frame binding each x_i to a_i 's value, evaluate B there on a private copy of the environment, and pop. The result is the type of the call. Each a_i must have type A_i , evaluated with the earlier parameters bound.

[Def] `fix f ( $\bar{x} : \bar{A}$ ) : B by  $x_j := b$` is checked at its *generic call*: from an environment with a fresh owned place $c_i \mapsto \sigma_i$ for each borrow parameter $x_i : \&T_i$, the call $f(\bar{a})$ with $a_i = \&c_i$ for borrow parameters, $a_i = *$ for proof parameters, and $a_i = \sigma_i$ otherwise; parameter types are evaluated left to right, with the earlier parameters bound, and stored with their bindings. The goal is the [Call-type] of this call. The body runs in the pushed frame, which is then dropped; its result's type must be convertible to the goal, as refined by the splits on the way.

[Split] When the checked program matches on a place whose content has an abstract head σ , each arm is checked with the refinement $\sigma := Z$, respectively $\sigma := S \sigma'$ for fresh σ' , applied to the environment, the goal and every stored type. If the head is a sealed program, every occurrence of it is first replaced by a fresh σ . If the match is followed by more code, that code is checked once, from the state in which the match has been closed off as a stuck block. A match on a proof is decided by the proof's type instead, with no refinement (Section 6.5).

[Rec] In the body of `fix f ... by  $x_j$` , including inside nested functions and block arms, f occurs only as the head of a call, and each recursive call passes in position j a value, or a borrow of a value, that is a strict subterm of x_j 's entry value σ_j as refined so far.

Figure 6: The typing rules particular to Ochr.

6.1 One rule for goals and induction hypotheses

A definition is checked by running it at its most general call [Def]. For `AddMZero( $x : \&\text{Nat}$ )` that is `AddMZero(&c)` from $\{ c \mapsto \sigma \}$: the parameter becomes a borrow of a fresh owned place, which stands for whatever the real caller lends. The goal is then computed by [Call-type], the rule that types every call.

The same rule types the recursive call in the successor case, at a different environment. There the argument $\&p$ borrows the predecessor field of c 's content, so evaluating the callee's statement at the call site observes c through the successor that surrounds the field. This is how the induction hypothesis arrives already wrapped in S . The generic call and the recursive call are two instances of one rule, which is why the goal and the hypothesis meet without a congruence step. That the callee's statement, proved at its generic call, remains true at every call site rests on the frame property (Section 7): a call affects only what it is passed, so the caller's observation is the callee's, placed in a context.

6.2 Types are formed once

A type is evaluated once, when it is formed, on a private copy of the environment, and becomes a closed statement about values. Later mutation cannot change it. This includes the codomain of a Π -type, which is a closure over the values of the variables it mentions; only its parameters are bound later, at each call. The goal of a definition is formed at its generic call, before the body runs, so a body that mutates its argument and then proves something about the new contents does not prove the goal: $\lambda(x : \&\text{Nat}). (*x := 5; \text{refl})$ does not have type $\Pi(x : \&\text{Nat}). \text{Id Nat } (*x) 5$.

6.3 Case splitting and joins

A match in the checked program on an abstract value cannot pick an arm, so each arm is checked with the abstract value refined [Split]. What follows the match is checked once, from a state in which the match itself

has been closed off as a stuck block. The closed-off match is precise: its sealed programs contain the match, and they reduce as soon as a later split decides it. There is no join of environments and no loss of information, and a borrow whose origin depends on the branch is handled like any other returned borrow. **AddToOne** of Section 2 is accepted, and so is any function whose body is split into a helper at a match: a match and a call to a function containing it are treated alike. Errors in the code after a match are reported once, at the line where they occur, not once per arm.

6.4 Erased terms leave no trace

Types and proofs are erased at runtime. The machine mirrors this exactly: it evaluates an erased term on a private copy of the environment, argument evaluation included, and discards the copy. A term is erased when it stands in a type position, when its declared type has sort **Prop**, or when it is a call to a function whose codomain term is a sort or has declared sort **Prop**; the decision never looks at a normal form. Two things follow. A formed type is a closed statement about values, as described above. And a proof can have no effect on the program around it: it may mutate places it creates itself, on its own copy. Running a proof and skipping it are therefore indistinguishable, and the machine skips proofs, which is what the compiled program does too.

Erased terms are also checked not to write, borrow or move a place that outlives them, except by handing it to another erased call. With the private copy this is redundant for a correctly classified term, and it is there as a partial fail-safe: if the two evaluation paths disagreed about whether an inline term is erased, the path that erases it would reject it rather than silently discard its effects. For calls and stuck blocks the fail-safe does not reach inside the callee’s body, and soundness rests on the classification being syntactic (Section 8). This is one principle, not a restriction on programs. **AddMZero**’s successor case borrows the field **p** to pass it to the induction hypothesis; the borrow happens on the private copy, and the real environment is untouched. Nothing in a program is marked pure, and any program may appear in a statement.

6.5 Proofs are matched by their type

False, **True** and **And** are inductive declarations, and a proof of one of them is matched like any other constructor value, with one difference: a proof cannot be inspected. Its value is *****, whether it is a parameter (bound to ***** at the generic call) or the result of an erased call, so there is no constructor to select an arm and no abstract value to refine. A match on a place whose type is an inductive $\mathbf{D}(\bar{a})$ of sort **Prop** is therefore decided by that type alone, never by the content. **False** has no constructors, so **match h {}** has no arms, and it is well typed at any result type: this is *ex falso*. **True** and **And** have one constructor, so the match has one arm, whose fields are places holding *****, of the field types ($l : P$ and $r : Q$ for $h : \mathbf{And}(P, Q)$). A declaration with several constructors, such as a user’s **Or**, gets one arm per constructor, each checked with its proof fields holding ***** and its data fields, if any, holding fresh abstract values; by the rule below every arm is then a proof, so the match is erased and never runs. The machine, which has no types, recognises a match on a proof from the constructors its arms name, which is a fact about declarations.

This keeps the two evaluation paths in agreement. The decision depends only on the scrutinee’s type being an inductive declared in **Prop**, and refinement never changes that: substituting for abstract values can change a type’s parameters, never its head. A type that is still a neutral, such as $\mathbf{Le}(\sigma, \sigma')$ before a split, does not license a match on a proof at all, so a match accepted at the generic call is decided the same way at every instance. A match with no arms is unreachable in a closed run. In a proof position it is erased like any proof; anywhere else, reaching it is stuck, which happens only when a refinement has made a hypothesis false, so it never yields a value of the wrong type.

Subsingleton elimination. A match on a proof may produce data or a type only if the inductive has no constructors, or one constructor whose fields are all proofs. **False**, **True** and **And** qualify. This is Lean’s rule [8]. With inductive $\mathbf{Or} (P Q : \mathbf{Prop}) : \mathbf{Prop} := \mathbf{Inl}(p : P) \mid \mathbf{Inr}(q : Q)$, the function $\mathbf{IsL}(h : \mathbf{Or}(\mathbf{True}, \mathbf{True})) : \mathbf{Bool} := \mathbf{match} \ h \ \{ \mathbf{Inl}(p) \Rightarrow \mathbf{true}, \mathbf{Inr}(q) \Rightarrow \mathbf{false} \}$ would mean **true** on $\mathbf{Inl}(\mathbf{refl})$ and **false** on $\mathbf{Inr}(\mathbf{refl})$, two proofs that proof irrelevance identifies. In *Ochr* this alone gives no closed proof of **False**: every proof is *****, so no match can see which constructor built one, and the closed proof appears only if proofs also keep their constructors. Subsingleton elimination is needed all the same. Without it **IsL** has no meaning in

the set model, where `Eq Bool (IsL(Inl(refl))) (IsL(Inr(refl)))`, provable by `refl`, would read `Eq Bool true false`; `IsL(Inl(refl))` would be a closed `Bool` that is stuck for ever, against canonicity; and a match with two constructors would have no arm to take unless every arm is a proof. For the inductives that qualify there is nothing to tell apart: the type determines the arm, and the fields are `*`.

6.6 Why each condition is there

Each side condition was added in response to a concrete counterexample: a closed proof of false, an accepted program that goes wrong when run, or a definition with no set-theoretic meaning. All but subsingleton elimination, which is Lean's, were found while designing the calculus. Each counterexample is a regression test in the implementation, and the ledger records what switching each condition off lets back in (Section 8). Most of them guard one invariant: a statement is evaluated once through closing off, at a definition's generic call, and again directly at each instance, and `[Call-type]` and `[Split]` equate the two, so every decision the two paths must agree on is made from syntax (Conjecture 3).

7 Metatheory

This section states the properties Ochr is designed to have and the evidence for each (Figure 8). Few of them are proved. A Lean mechanisation proves, for a first-order fragment of an earlier version of the rules, that machine

Condition	What goes wrong without it
Π -types capture values	<code>h : Π(_ : Unit). Id Nat x Z</code> proved when <code>x</code> was <code>Z</code> is re-read after <code>x := S x</code> , proving <code>Id Nat (S Z) Z</code> .
Recursion on entry values; <code>f</code> only as a call head; no <code>f</code> without <code>by</code>	<code>*x := S *x; match *x { S p => f(&p) }</code> recurses on the original value; passing <code>f</code> to a helper or calling it in a <code>by</code> -less body avoids the check; each proves <code>False</code> .
Erased terms run on a private copy, and may not affect outside places	Proof irrelevance identifies <code>$\lambda x. *$</code> with <code>$\lambda x. (*x := 7; *)$</code> ; skipping only proof <i>calls</i> lets a closed-off proof block and the same block run inline disagree.
Erasure read from syntax, never from normal forms; universes not cumulative	A call <code>W(&c, n) : U(n)</code> with <code>U(n) := match n {Z => Prop, ...}</code> runs at the generic <code>n</code> but is erased at <code>n = Z</code> ; the same for a local function whose codomain mentions captured values, and for a type-valued match erased only when closed off.
Functions compared by their observation	Comparing results alone identifies <code>$\lambda x. (*x := S Z)$</code> with <code>$\lambda x. ()$</code> ; comparing borrow-returning functions without writing through the result identifies <code>$\lambda(x, y). x$</code> with <code>$\lambda(x, y). y$</code> .
Pattern variables are places	A stuck block that writes through <code>p</code> in <code>match *x { S p => p := Z }</code> captures <code>*x</code> by copy.
All owners observed	A borrow returned into one of two arguments leaves its hole in both; observing one proves <code>Eq Nat 0 1</code> .
Exclusive access; loan-free closing off; matches end loans in neutral heads	A live loan copied into a sealed program, or a hole generalised away by a split, lets an accepted program write through an ended borrow.
Generalisations are global; fresh names never reused	A generalisation made while forming a type is lost with its private copy, and its name is reissued for a different computation.
Borrows only of data, and <code>&</code> only at the top of a declared type	<code>$\Pi(x : \&\text{Type}_e)(a : *x). *x : \text{Type}_e$</code> makes <code>Type_e</code> impredicative, embedding System U; a codomain that computes to <code>&Nat</code> is closed off as data at the generic call but returns a live borrow at an instance, and an accepted program reads a moved borrow.
A borrow-returning function type takes a borrow	A call of <code>g : $\Pi(n : \text{Nat}). \&\text{Nat}$</code> returns a borrow observed by no owner, so Ochr refutes a type that safe Rust inhabits with a leaked <code>'static</code> borrow, and any opaque inhabitant proves <code>False</code> .
Strict positivity	<code>inductive Bad := Mk(f : $\Pi(x : \text{Bad}). \text{False})$</code> gives a closed proof of <code>False</code> that is never run.
Subsingleton elimination	<code>match h { Inl(p) => true, Inr(q) => false }</code> on <code>h : Or(True, True)</code> has no set-theoretic meaning and leaves <code>IsL(Inl(refl))</code> a closed <code>Bool</code> stuck for ever; if proofs also kept their constructors, it would prove <code>False</code> .

Figure 7: The side conditions of Ochr and the counterexamples that forced them.

Property	Status	Evidence
1. The machine is deterministic, monotone in fuel, and invariant under renaming of loans.	mechanised	<code>eval_det</code> , <code>exec_mono</code> , <code>exec_rename</code>
2. Machine steps preserve well-formedness (Section A.5).	mechanised	<code>exec_wf</code> , for source terms that do not name the machine’s argument temporaries
3. Concrete runs of programs accepted by [Rec] terminate.	partly mechanised	for borrow-free and for non-recursive programs; in general <code>sorry</code>
4. Frame: a call affects only what it is passed, and its effect is its isolated run, plugged back into the caller.	mechanised	<code>frame_local</code> , <code>call_effect</code>
5. The sealed programs of [Close] compute the call’s result, the final contents of its borrowed places and its backward function.	mechanised, with hypotheses	<code>close_res</code> , <code>close_fin</code> , <code>close_cur</code> , <code>close_back</code> ; the hypotheses follow from 2, not yet in Lean
6. A backward function is injective, and so is the caller’s context around it when every owner of the hole is observed; with fewer owners it can be constant.	mechanised (machine form)	<code>back_inj</code> , <code>ctx_inj</code> , <code>ctx_needs_all_owners</code>
7. The order in which borrows end does not change the resolved state.	partly mechanised	two endings commute (<code>end_comm</code>); tests of every order
8. Stability: the two evaluation paths take the same decisions (Conjecture 3).	conjecture	the counterexamples of Figure 7 and the ledger (Section 8)
9. Naturality and adequacy: refining and running commute up to resolution, so an <code>Id</code> computed on abstract inputs holds on every concrete input.	conjecture	tests; the form without resolution is false (mechanised counterexample)
10. Consistency: no closed term has type <code>False</code> (Conjecture 2).	conjecture	the model sketch below; the regression suite

Figure 8: What Ochr is meant to satisfy, and the evidence. “Mechanised” means proved in Lean without `sorry`, for the fragment of Section 7.3.

steps preserve well-formedness, that a call affects only what it is passed, and that sealed programs compute the forward and backward functions of Aeneas (Section 7.3). Consistency, the agreement of the checker’s two evaluation paths, and adequacy are conjectures; for them we give the model in which we expect to prove them, and the argument we expect, as sketches. The rest of the evidence is empirical: every counterexample found while designing the calculus is a regression test of the checker, and its counterfactual ledger shows which rule each test depends on (Section 8).

7.1 The model, as a sketch

Following Aeneas [15], a computation is read as a pure function from the current contents of its borrows to its result and their final contents; types get a set-theoretic interpretation after Carneiro [8].

What consistency would be relative to. Computations translate into CIC_L , the intensional type theory of Lean 4 [24], with an impredicative, definitionally proof-irrelevant `Prop` and `propext`. Types need more: Ochr’s conversion identifies propositions that CIC only proves equivalent, through the `Eq` rules (Section 5) and through [Call-type], so no translation into intensional CIC preserves conversion. We therefore interpret types directly in sets, following Carneiro’s construction, with one inaccessible cardinal per universe level. Carneiro needs unique typing to decide whether a λ or Π denotes a proof or a set-theoretic function; in Ochr that decision is syntactic (Section 6), so the interpretation can be defined by recursion on derivations. Borrows are only of data (Section 3): a borrow of a type would let $\Pi(x : \&\text{Type}_\circ)(a : *x)$. $*x$ live in $\text{Type}_\circ$, which is then impredicative and has no set model.

Types. An inductive declaration in $\text{Type}_\circ$ denotes the least set closed under its constructors, at each value of its parameters, so `Nat` denotes $\mathbb{N}$; constructors are injective and have disjoint images. One in `Prop` denotes a subsingleton, $\{\bullet\}$ if that least set is inhabited and $\emptyset$ otherwise: `False` denotes $\emptyset$, `True` denotes $\{\bullet\}$, and `And(P, Q)` denotes $P^\dagger \cap Q^\dagger$. `Eq A a b` denotes $[a^\dagger = b^\dagger] \subseteq \{\bullet\}$. For a function type with borrow parameters $x_i : \&T_i$ at positions $i \in I$, let $D_i = T_i^\dagger$ for $i \in I$, $D_j = A_j^\dagger$ otherwise, and $\text{Fin} = \prod_{i \in I} T_i^\dagger$, the final contents of the borrowed places:

$$(\Pi(\bar{x} : \bar{A}).B)^\dagger = R_{\Pi(\bar{x}:\bar{A}).B} \subseteq \Pi(\bar{d} : \bar{D}).\text{Out}_B(\bar{d}), \quad \text{Out}_B(\bar{d}) = \begin{cases} B^\dagger(\bar{d}) & \text{if } B \text{ is a proposition,} \\ B^\dagger(\bar{d}) \times \text{Fin} & \text{if } B \text{ is borrow-free data,} \\ T^\dagger \times (T^\dagger \rightarrow \text{Fin}) & \text{if } B = \&T. \end{cases}$$

Here $B^\dagger(\bar{d})$ is B at the generic call of [Def]. We write fwd_F and back_F for the two components of $F(\bar{d})$. For a returned borrow, $\text{back}_F(\bar{d})$ is Aeneas's backward function: it maps the value the caller eventually leaves in the borrow to the final contents. A proposition has no Fin component, because proofs run on a private copy (Section 6).

Definition 1 (injectivity relation). $R_A \subseteq A^\dagger$ is defined by induction on A . For base types, propositions and universes $R_A = A^\dagger$; R is componentwise on pairs, and $R_{\&T} = R_T$. For a function type, $F \in R_{\Pi(\bar{x}:\bar{A}).B}$ iff, for all $\bar{d}$ with $d_i \in R_{T_i}$ for $i \in I$ and $d_j \in R_{A_j}$ otherwise, $\text{fwd}_F(\bar{d}) \in R_B$, and, if $B = \&T$, $\text{back}_F(\bar{d})$ is injective: equal tuples of final contents come only from equal final values of the returned borrow.

R is forced, and it is where Ochr's function spaces part from Rust's. By [Call-type], Ochr proves of every abstract $g : \Pi(x : \&\text{Nat}). \&\text{Nat}$ that writing 0 or 1 through $g(x)$ leaves different states, which is false of a Rust function that returns a leaked 'static borrow. Ochr has no 'static borrows, and an opaque definition of a borrow-returning type is an injectivity assumption (Section 10).

Terms. Abstract values become variables, and a loan ℓ becomes a variable h_ℓ , a *hole* for the final content of its borrow. The *view* $\Omega^\dagger$ reads a borrow as its current content and a loan as its hole; the *resolution* ρ_Ω substitutes each hole by its borrow's content, that is, it ends every borrow. A term translates to a state-passing function on views (Figure 9); a sealed program denotes the result of running it from the empty view; a definition is a well-founded recursion on the entry value of its decreasing parameter; and Id is the equation between the tuples of result and resolved footprint. A symbolic and a concrete run of one term can place their borrows differently (Section 7.2), so runs are related through their resolutions.

$$\begin{aligned} \text{End}_\ell : s &\mapsto (s \setminus \text{holder}(\ell))[h_\ell := s.\text{holder}(\ell)] \\ \text{match } p\{\dots\} : s &\mapsto \text{natCase } (s.p) \left(t_Z^\dagger(s[p := 0]) \right) \left(\lambda\sigma'. t_S^\dagger(s[p := \text{succ } \sigma']) \right) \\ f(\bar{a}), B \text{ data} : s &\mapsto \text{let } (r, \bar{\varphi}) = f^\dagger(\bar{u}, \bar{w}) \text{ in } (r, (s \setminus \bar{a})[h_{\ell_i} := \varphi_i]) \\ f(\bar{a}), B = \&T : s &\mapsto \text{let } (c, \beta) = f^\dagger(\bar{u}, \bar{w}) \text{ in } (c, (s \setminus \bar{a})[h_{\ell_i} := \beta_i(h_k)]) \end{aligned}$$

Figure 9: Key clauses of the translation. $\bar{u}$ are the contents of the borrow arguments, whose loans are ℓ_i , and $\bar{w}$ the other arguments; k is fresh. Ending a borrow is a substitution, and a call is one application, with Aeneas's region abstraction as the substitution $h_{\ell_i} := \beta_i(h_k)$. An erased term leaves the view unchanged.

Four invariants of the machine make this well defined (Section A.5): each borrow is held once, every loan is bound by a held borrow or lies inside a sealed program (the only place a loan may occur twice), loans nest in borrows acyclically, and every value read, moved or passed is loan-free. That machine steps preserve them is property 2 of Figure 8, which is mechanised.

Soundness, as a conjecture. For a program accepted by [Def] and [Rec] and every valuation of its abstract values, we conjecture that types of sort s denote elements of s 's universe, that convertible types denote the same set, that every term denotes a function from views before its run to its type's denotation and views after it, and that every definition lies in R . The proof we expect is by induction on derivations, with most cases reading the translation backwards. The **Eq** rules hold because propositions with the same truth value are equal; injectivity and disjointness are instances, since constructors are injective and have disjoint images (in CIC_L , **propext** applied to the no-confusion property of constructors). A match on a proof is interpreted through the proof's type, which subsingleton elimination makes well defined. The hardest case is [Call-type], which needs the frame property (property 4) extended to types, the injectivity of contexts (property 6) and naturality (property 9).

Conjecture 2 (consistency). Relative to ZFC with one inaccessible per universe level, no closed term has type **False** (nor, therefore, **Eq Nat Z (S Z)**, which computes to it).

Whether type checking is decidable is also open: types may combine large elimination, **J** with type-valued motives and an impredicative, proof-irrelevant **Prop**, which Abel and Coquand show can defeat normalisation [1]. The model interprets derivations, so consistency does not depend on it.

7.2 The two paths agree

A statement is evaluated along two paths: at a definition’s generic call, where calls and matches on abstract values close off, and directly at each instance. [Call-type] and [Split] identify the results, so any decision taken differently on the two paths is an inconsistency, and most of Figure 7 is a list of such decisions. A *refinement* α substitutes values for abstract values, definable closures for abstract functions, and values for holes; it covers both case splits and instantiation at a call site.

Conjecture 3 (stability). For t at Ω and a refinement α , the following are the same for t at Ω and $t\alpha$ at $\Omega\alpha$: (1) which subterms are erased, and which matches are decided by the type of a proof; (2) which places each closed-off stuck block captures, in which modes, and the places of each footprint; (3) whether each match’s arms are well typed; (4) each call’s [Close] row.

The argument we expect is that each decision is read from syntax or declarations, and α changes only normal forms. (1) A call is erased iff its callee’s codomain *term* is a sort or has declared sort **Prop**, a stuck block iff each of its arms is, universes are not cumulative, and a match on a proof needs its scrutinee’s type to have an inductive head declared in **Prop**, which α cannot change. (2) Pattern variables are resolved to sub-places before captures and footprints are computed. (3) Arms are checked under every refinement of the scrutinee, inside types too. (4) The row is read from the declared codomain, and $\&$ occurs only at the top of a declared type. Most of these were once decided from a normal form, with a closed proof of **False** or an accepted program that goes wrong as the result (Figure 7). The argument is an enumeration of the decisions we know of; the last round of review found one we had missed, a codomain that computes to a borrow type, which is why (4) now forbids it. The failure it rules out resembles what Pédrot and Tabareau, discussing call-by-name, call a desynchronisation between the effects performed in a term and in its type [29]; we claim only the resemblance.

Naturality holds only up to resolution, which costs completeness. With `Pick(n, x, y) := match n { Z => x, S _ => y }` and n abstract,

```
let r = Pick(n, &a, &b); let z = b; match n { Z => *r := 5, S _ => () }
```

runs for every concrete n but is rejected: `Pick`’s hole sits in the fills of both `a` and `b`, so reading `b` ends r symbolically, and r is dead in the `Z` arm. The refined symbolic state and the concrete one agree only after every borrow is ended (a mechanised counterexample to the stronger form), which is why property 9 is stated up to resolution; Rust likewise treats r as borrowing both places while it is live.

7.3 What is mechanised

The directory `ochr/core/meta-lean` of the artifact (about 11,600 lines of Lean 4, no Mathlib) mechanises the effectful layer of version 1.3 of the rules, which predates the erasure and inductive-proposition rules of Section 6, for a first-order fragment: natural numbers, unit, pairs, places, borrows, matching, recursive definitions (including borrow-returning ones), opaque definitions and erased proofs, with no closures, types, inductive propositions or stuck blocks (stuck blocks are lambda-lifted by hand). The machine is a clocked big-step interpreter. A plain `lake build` checks every module, and the theorems that Figure 8 marks as mechanised are proved there using only Lean’s standard axioms. The frame property (property 4) and the [Close] equations (property 5) are equations that hold at every amount of fuel; the [Close] equations take as hypotheses facts about loan-freedom that follow from well-formedness, but are not yet derived from it in Lean. Well-formedness (property 2, `exec_wf`) is preserved by every run of a source term that does not name the machine’s argument temporaries, which no surface program can; without that side condition it is false. One statement remains `sorry`: termination in general (**termination**). Nothing about types, conversion, stability or the model is mechanised. It also checks, as a test, that naturality fails without resolution (the `Pick` example above, in `Tests/Basic.lean`).

8 Implementation

We implemented the calculus as an executable reference checker in Lean 4, about 4,000 lines with no dependencies, plus about 3,500 lines of examples and tests. Its purpose is not performance but fidelity: a hand derivation can be charitable where the rules are silent, and a program cannot. Every example in this paper is checked by it, written in a concrete syntax close to the one used here:

```
ochr Numbers {
  def AddM (x : &Nat) (y : Nat) : Unit by x :=
    match *x { Z => *x := y, S p => AddM(&p, y) }

  def AddMZero (x : &Nat) : Id Unit (AddM(x, 0)) () by x :=
    match *x { Z => refl, S p => AddMZero(&p) }

  reject def WriteThenRefl (x : &Nat) : Id Nat (*x) 5 := *x := 5; refl
}
```

A **def** is expected to be accepted and a **reject def** to be rejected, and each verdict is an assertion checked during the build, so a build succeeds only if every verdict is as expected. Each test file also asserts how many verdicts it contains, which guards against a test file that silently checks nothing.

Structure. The machine, closing off, sealed-program normalisation, splitting, closing off stuck blocks, observation, **Id**, [Call-type], [Def] and [Rec] form one mutually recursive block of about two thousand lines. Terms use de Bruijn indices, and the variables introduced by [Close] have fixed names, so “same normal form” is structural equality, except that function values and Π -types are compared at their generic arguments and the unit laws of **And** are applied. The implementation follows the rules closely enough that its trace of **AddMZero** reproduces the derivation of Section 2 symbol for symbol, including the successor that the environment supplies to the induction hypothesis.

Test suite. The suite contains every program this paper prints, each mapped to its test in the artifact’s notes: **AddM**, **AddMZero**, **Add** and **AddZero**; the returned-borrow examples **TailM**, **AddM'** and **AddMEq**; branching with borrows whose origin depends on the branch; opaque function arguments; the programs of Section 2 that pass a proof about the current, mutated state; user-declared inductive types, among them in-place append on a polymorphic **List(A)** and the binary-search-tree insertion of Section 2 with its size theorem; and the logic of Section 3: **ex falso**, disjointness, matches on proofs, and the **Or** attack of Section 6.5. It also contains every program that must be rejected, in particular every closed false proof, and every accepted program that went wrong, found while designing the calculus. All 440 verdicts are as expected. The compiled checker decides all of them in about 19 ms; a clean build takes about 55 s, most of it re-running the suite for each row of the counterfactual ledger.

The counterfactual ledger. Each rule that is not the evident typing of a form has a switch that turns it off. For each of the 47 switches, the build asserts exactly which verdicts flip when it is off, and the class of the row, which follows from what flips:

- *Soundness*, 23 rows: with the rule off, the checker accepts a closed proof of **False** (17 rows) or an open program that goes wrong when run (6 rows), and the row names these witnesses. Without entry-value recursion, the non-terminating “proofs” of Section 6 are accepted; without exclusive access, an accepted program writes through an ended borrow; without the rule that a recursive function occurs only as the head of a call, a function that passes itself to a helper proves **False**; and without owner sets (Section 5), a function that returns a borrow into one of two arguments yields a closed proof of **False**.
- *False lemma*, 1 row: without reading erasure from declarations, the checker accepts a false statement, whose closed instance confinement now rejects.
- *Model*, 3 rows: with the rule off, the checker accepts definitions that have no set-theoretic meaning, though the suite contains no closed proof of **False** from them. These are the borrow parameter of a borrow-returning function type, subsingleton elimination (Section 6.5), and borrows only of data.

- *Policy*, 3 rows: with the rule off, the checker accepts only programs that are true under the other rules. These are reading [Close]’s row from the declared codomain, which stability needs; confinement; and classifying a variable as a proof by its declaration when confinement is also off.
- *Completeness*, 16 rows: with the rule off, the checker only rejects good programs. Among them are the private copy for erased terms, generalising a sealed program before splitting on it, binding proof parameters to $\star$, [Seal]’s head guard, matching on a proof by its type, and the unit laws as conversion. One of them switches *on* an extension, confining the bodies of functions that return proofs or types, to record what it would cost.

No row flips nothing. Confinement backs up two other rules: the soundness witnesses of the private copy and of reading proofs by their declaration appear only when confinement is off too. The ledger shows that each rule is needed for some verdict, not that the rules suffice, and one side condition has no switch: non-cumulative universes.

What running the rules found. Writing the checker exposed gaps in earlier versions of the rules, each now fixed: the recursive-function-as-a-value loophole above; an argument evaluated earlier in the same call being invalidated by a later one ($f(\&x, x)$ ends the first borrow while reading the second argument); recursive calls inside nested closures, which must be checked against the entry values of the enclosing function; with trees, the need to apply a split on a sealed program to re-derivations of it as well; and, with the connectives as declarations, that the unit laws must be conversion rather than a rewriting of stored types, or a proof of $\tau \wedge P$ could not be matched. It also found a bug in an earlier version of the checker itself: a match’s scrutinee was typed from its arms, so a match on a number could run a list’s arms. The counterexamples summarised in Figure 7, including several that an earlier version of the checker accepted, are regression tests, and each soundness row of the ledger flips at least one of them.

9 Related work

Functional translation of borrows. Aeneas [15] translates safe Rust into a pure functional program in Lean, F* or Coq. Its symbolic semantics of the low-level borrow calculus is the direct ancestor of our machine: we take its values with loans and borrows, its lazy ending of borrows, and its insight that a mutable borrow parameter is a forward computation plus a backward function. Later work gives that symbolic semantics joins and loops and proves it sound as a borrow checker [16]. The difference is where and when the translation happens. Aeneas translates eagerly, whole-program, once, and the user reasons about the translated artifact, where backward functions are named definitions. Ochr never produces a translated program. The same state-passing content appears lazily inside conversion, only as far as a particular equation needs, and only in source syntax: a sealed program is what Aeneas would call an application of a backward function. Aeneas is modular in that it never unfolds a callee; Ochr must unfold, because unfolding is what definitional equality is, and closing off is what keeps the unfolding finite.

Prophecies. RustHorn [23], RustHornBelt [22] and Creusot [11] model a mutable borrow as its current value paired with a *prophecy* $\wedge x$ of its final value, resolved when the borrow ends. A filled loan in Ochr is a prophecy that has been resolved by the machine, and the hole in a sealed program for a returned borrow is an unresolved one. Prophecies are logic variables introduced at borrow creation; Ochr introduces abstract values only where the checker needs an unknown, never at a borrow, which is what keeps concrete evaluation free of symbols (Section 7).

Specification-based verification of imperative code. Verus [18], Creusot, Prusti [6], Flux [19] and VeriFast [17] verify imperative code against specifications written separately, in pre- and postconditions, refinement types or separation logic, and discharge the obligations with automated provers. Hoare type theory [25] and Ynot [26] internalise the triples in a dependent type theory, as do F*’s separation-logic DSLs Steel and Pulse [12]; F*’s Dijkstra monads [3] internalise weakest preconditions and let **reify** turn a stateful computation into a state-passing function. Relational reasoning about two effectful programs is possible in some of these, notably through F*’s **reify**, which exposes a stateful computation as a state-passing function that can then be compared; the comparison is about the reified pure function. In Ochr the specification of an in-place function is another

program, or a theorem about the function itself, equality of computations is a type, and it is decided on the effectful code by conversion where possible.

Effects in dependent type theory. The fire triangle [29] shows that substitution, dependent elimination and observable effects cannot coexist consistently, and ∂ CBPV organises the possible compromises. Closed Ochr programs are not observably effectful in their sense (Section 1.3). For open terms Ochr takes the call-by-value way out, keeping dependent elimination and restricting substitution to values, which Pédrot and Tabareau illustrate with PML. Lepigre’s PML [20] relaxes the syntactic value restriction to a semantic one: an argument need only be observationally equivalent to a value. Ochr’s typing instead runs the argument and substitutes the value it computes. The call-by-value type theory that Pédrot and Tabareau obtain by translation into ∂ CBPV has no dependent **let** and no large elimination, its types being syntactic values; Ochr computes its types by running the checked program, and has both. Zombie [9] combines a logical fragment with a call-by-value programming fragment that allows general recursion: its types may mention arbitrary programs, an equation between programs is proved by evaluating both sides to a common term, and several of its typing rules apply only to values. Ochr shares the evaluation-based equality and the attention to values, for programs that are total but mutate their environment. Ahman’s eMLTT [4] equips computations with definitional equations from an algebraic theory of global state; Ochr’s state is structured, local and borrow-shaped, and its equations come from running the program rather than from an axiomatised theory.

Observational equality. Observational type theory [5, 28] computes the equality type by the structure of the type: equality of pairs is a pair of equalities, equality of functions is pointwise. Ochr’s **Eq** computes by the structure of constructor values (injectivity and disjointness), and **Id** applies the idea at the type of observations, which is why **Eq** stays primitive rather than an inductive family. Unlike OTT, Ochr compares functions by their generic observation rather than pointwise, and its cast **J** computes only on convertible endpoints. Pujet and Tabareau’s impredicative OTT [27] studies the same combination of an impredicative, definitionally proof-irrelevant **Prop** with a computing equality, and Gilbert et al. [14] show why an eliminator of a definitionally irrelevant equality must not reduce on open terms, which is the reason for Ochr’s rule for **J**.

Inductive propositions. Ochr’s connectives are inductive declarations in **Prop**, as in Lean (**inductive False : Prop**), Coq (**Inductive False : Prop := .**) and Agda (**data $\perp$: Set where**), and matching on a proof follows Lean’s subsingleton elimination [8]. Encodings such as **False := Π (P : Prop). P** belong to calculi without inductive types, such as the pure calculus of constructions [10]. Indexed families, and with them equality as an inductive family, are future work.

Pure languages with in-place updates. Lean 4 performs destructive updates on values it can prove unshared at runtime [30], and Koka’s fully in-place fragment [21] guarantees statically that a function allocates nothing. Programs in these languages are pure, so their proofs are ordinary, but the programmer writes functional code and cannot express a borrow that is returned to the caller. Linear and quantitative type theories [7] and ATS [31] track resources in types; Ochr uses the same separation between runtime usage and erased type-level usage as quantitative type theory, where a variable used in a type is not consumed.

Relational reasoning. ReLoC [13] and relational Hoare logics prove two programs equivalent inside a program logic. Ochr’s **Id** is a relational statement about two computations, but it is decided, where possible, by conversion, and it lives in the same type theory as the programs.

10 Discussion

10.1 What generalises directly

Inductive types. The body illustrates the rules on natural numbers, but nothing in them is specific to numbers: data and the logical connectives are declared the same way (Section 3), and the implementation supports user-declared inductive types (Section 2, Section 8). A constructor with several fields gives several sub-places, **[Split]** refines an abstract value to a constructor applied to fresh abstract values, and **[Rec]** asks for a strict subterm of

the entry value. Indexed families are future work; with them, **Eq** could be declared rather than built in, provided its observational computation rules survive the change.

Opaque definitions. A definition may be declared with a type and no body, or used through an abstract function value. A call to it is stuck at once and closed off, so its effects are recorded as sealed programs mentioning it, and every theorem about its callers remains available. This is how *Ochr* recovers the modularity of Aeneas’s symbolic execution [15], which never unfolds a callee: the programmer chooses, per definition, whether conversion may look inside. An opaque definition that returns a borrow, however, carries an assumption (below).

Shared borrows. Immutable borrows are simpler than mutable ones in this setting: a shared borrow never has a final value different from its initial one, so closing off a call that takes one needs no sealed program for it. We omitted them to keep the core small.

10.2 The frontier

Borrows stored in data. The central restriction of the core is that borrow types never occur inside other types. It rules out an iterator that returns a list of borrows into a list, $\text{IterM} : \&(\text{List Nat}) \rightarrow \text{List } (\&\text{Nat})$. There, the number of borrows returned is itself unknown on symbolic input, so no fixed set of holes can stand for their final values. The natural generalisation is a hole that stands for a whole data structure of final values, filled when the last borrow in the structure ends, which is the neutral form of an Aeneas region abstraction over a symbolic value with a borrow projector. The same restriction excludes closures that capture borrows.

No 'static borrows. Every borrow a function returns derives from one of its borrow arguments, and a function type returning $\&\text{T}$ must take a borrow. This is Rust’s lifetime-elision rule, but in *Ochr* it is a restriction, not a default. Safe Rust lets a function of type $\text{fn}(\&\text{mut u64}) \rightarrow \&\text{mut u64}$ return $\text{Box}::\text{leak}(\text{Box}::\text{new}(0))$, a 'static borrow unrelated to its argument, whose backward function is constant; *Ochr* proves that every function of type $\Pi(x : \&\text{Nat}). \&\text{Nat}$ has an injective backward function (Section 7.1), which is false of that one. So *Ochr*’s borrow-returning function types are Rust’s without 'static results, and an opaque or foreign definition of such a type is an unchecked assumption that its backward function is injective: neither *Ochr* nor Rust’s type discipline discharges it.

Borrows of an arbitrary type. A borrow must be of data, and whether a type is data is read from its evaluated head, so a function cannot take $\&\text{A}$ for a type parameter $\text{A} : \text{Type}_\circ$: A might be instantiated at a universe, a Π -type or a proposition, and borrowing a type makes $\text{Type}_\circ$ impredicative (Figure 7). A generic in-place swap is therefore not expressible. A separate universe of data types, below $\text{Type}_\circ$, would bring it back.

Loops. A loop is a tail-recursive local function, and its type is its invariant. Closing off a loop that is stuck on an abstract value is closing off that function. We have not designed the surface form, nor checked whether the invariants that arise are the ones programmers expect to write.

Interior mutability and unsafe code. Closing off is sound because a call can affect only what it is passed. Global state, interior mutability or unsafe aliasing would each break this frame property, and with it the rule. They would need to be confined, as Rust confines unsafe code behind safe interfaces, and given sealed-program semantics through their interfaces.

10.3 Costs

Checking cost. Type checking runs programs. On concrete data this is the cost of running them; on symbolic data each stuck call is closed off once, and each refinement re-runs the sealed programs that mention the refined value. Sealed programs are closed and canonical, so their normal forms can be cached and shared, and a refinement only re-runs what mentions it. Our implementation does no caching and decides every example of this paper in milliseconds (Section 8); we have not measured large programs.

The checked machine is not yet the compiled program. The machine that the checker runs, and that the proofs are about, copies borrow-free data on every read, where a compiled program should move it; an affine usage discipline on runtime code, outside the core, would license implementing each last-use copy as a move, and only then does *AddM* allocate nothing. The machine also evaluates types on a discarded private copy and skips proofs, where a compiled program would omit both. We have not defined the affine discipline, nor related

a compiled program to the machine. Both steps are meant to be unobservable, since a copy that is never used again cannot be told apart from a move, and an erased term’s run cannot be told apart from skipping it, but neither is proved: that the compiled program behaves as the checked one is, for now, an assumption. The gap is narrower than the one Section 1.1 criticises in the pure route: where the program mutates in place is written in its text and checked, and only the copying of values it never uses again is left to the compiler. But it is a gap. Types are exempt from the affine discipline, which is what lets $x + x = 2 \cdot x$ be stated for a type without a copy operation; this is the same separation between runtime usage and erased usage as quantitative type theory [7].

What the programmer sees. Goals and hypotheses are propositions about values, and every value that appears in them is either data, an abstract value named after the parameter it came from, or a sealed program, which is source code. A sealed program can be printed as the user’s own wrapper when one exists (`[let c = o; AddM(&c, 0); c]` is `Add(x, 0)`). Nothing in a goal mentions loans, borrows, environments or backward functions. We regard this as a design invariant, and it is a useful test for extensions: a feature whose goals cannot be printed as source code is one the programmer would need a new concept to reason about.

11 Conclusion

The usual way to verify an efficient imperative program is to write it twice. We have shown that, for programs whose mutation is disciplined by Rust-style ownership, the second copy can be generated on demand, inside the type checker, as source code: when definitional equality meets a call it cannot run, it closes the call off into sealed programs that own their state, and a later case split runs them again. Combined with an observational equality that compares computations by their results and their writes, this lets theorems about in-place code be stated about that code and proved by the same structural recursion one would use for pure code. Often the in-place proof is the simpler one, because the borrow structure of the environment performs the congruence steps a pure proof must spell out.

The core calculus is uniform, with data and the logical connectives declared the same way, and every side condition in it is there because a smaller one admitted a proof of false. The main open direction is borrows stored inside data, where the neutral form of a stuck call must describe a whole structure of final values rather than a fixed set of places.

Bibliography

- [1] Andreas Abel and Thierry Coquand. 2020. Failure of Normalization in Impredicative Type Theory with Proof-Irrelevant Propositional Equality. *Logical Methods in Computer Science* 16, 2 (2020).
- [2] Andreas Abel. 2013. Normalization by Evaluation: Dependent Types and Impredicativity. In *Habilitation thesis, LMU Munich*, 2013.
- [3] Danel Ahman, Catalin Hritcu, Kenji Maillard, Guido Martínez, Gordon Plotkin, Jonathan Protzenko, Aseem Rastogi, and Nikhil Swamy. 2017. Dijkstra Monads for Free. In *POPL*, 2017.
- [4] Danel Ahman. 2018. Handling Fibred Algebraic Effects. In *Proc. ACM Program. Lang. (POPL)*, 2018.
- [5] Thorsten Altenkirch, Conor McBride, and Wouter Swierstra. 2007. Observational Equality, Now!. In *PLPV*, 2007.
- [6] Vytautas Astrauskas, Peter Müller, Federico Poli, and Alexander J. Summers. 2019. Leveraging Rust Types for Modular Specification and Verification. In *Proc. ACM Program. Lang. (OOPSLA)*, 2019.
- [7] Robert Atkey. 2018. Syntax and Semantics of Quantitative Type Theory. In *LICS*, 2018.
- [8] Mario Carneiro. 2019. The Type Theory of Lean.
- [9] Chris Casinghino, Vilhelm Sjöberg, and Stephanie Weirich. 2014. Combining Proofs and Programs in a Dependently Typed Language. In *POPL*, 2014. 33–46.

- [10] Thierry Coquand and Gérard Huet. 1988. The Calculus of Constructions. *Information and Computation* 76, 2–3 (1988), 95–120.
- [11] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. 2022. Creusot: A Foundry for the Deductive Verification of Rust Programs. In *ICFEM*, 2022.
- [12] Aymeric Fromherz, Aseem Rastogi, Nikhil Swamy, Sydney Gibson, Guido Martínez, Denis Merigoux, and Tahina Ramananandro. 2021. Steel: Proof-Oriented Programming in a Dependently Typed Concurrent Separation Logic. In *Proc. ACM Program. Lang. (ICFP)*, 2021.
- [13] Dan Frumin, Robbert Krebbers, and Lars Birkedal. 2018. ReLoC: A Mechanised Relational Logic for Fine-Grained Concurrency. In *LICS*, 2018.
- [14] Gaëtan Gilbert, Jesper Cockx, Matthieu Sozeau, and Nicolas Tabareau. 2019. Definitional Proof-Irrelevance without K. *Proc. ACM Program. Lang.* 3, POPL (2019), 3:1–3:28.
- [15] Son Ho and Jonathan Protzenko. 2022. Aeneas: Rust Verification by Functional Translation. In *Proc. ACM Program. Lang. (ICFP)*, 2022. 116:1–116:31.
- [16] Son Ho, Aymeric Fromherz, and Jonathan Protzenko. 2024. Sound Borrow-Checking for Rust via Symbolic Semantics. In *Proc. ACM Program. Lang. (ICFP)*, 2024.
- [17] Bart Jacobs, Jan Smans, Pieter Philippaerts, Frédéric Vogels, Willem Penninckx, and Frank Piessens. 2011. VeriFast: A Powerful, Sound, Predictable, Fast Verifier for C and Java. In *NASA Formal Methods (NFM)*, 2011.
- [18] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs using Linear Ghost Types. In *Proc. ACM Program. Lang. (OOPSLA)*, 2023.
- [19] Nico Lehmann, Adam T. Geller, Niki Vazou, and Ranjit Jhala. 2023. Flux: Liquid Types for Rust. In *PLDI*, 2023.
- [20] Rodolphe Lepigre. 2016. A Classical Realizability Model for a Semantical Value Restriction. In *ESOP*, 2016. 476–502.
- [21] Anton Lorenzen, Daan Leijen, and Wouter Swierstra. 2023. FP^2 : Fully in-Place Functional Programming. In *Proc. ACM Program. Lang. (ICFP)*, 2023.
- [22] Yusuke Matsushita, Xavier Denis, Jacques-Henri Jourdan, and Derek Dreyer. 2022. RustHornBelt: A Semantic Foundation for Functional Verification of Rust Programs with Unsafe Code. In *PLDI*, 2022.
- [23] Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. 2020. RustHorn: CHC-based Verification for Rust Programs. In *ESOP*, 2020.
- [24] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *CADE*, 2021.
- [25] Aleksandar Nanevski, Greg Morrisett, and Lars Birkedal. 2006. Polymorphism and Separation in Hoare Type Theory. In *ICFP*, 2006.
- [26] Aleksandar Nanevski, Greg Morrisett, Avraham Shinnar, Paul Govereau, and Lars Birkedal. 2008. Ynot: Dependent Types for Imperative Programs. In *ICFP*, 2008.
- [27] Loïc Pujet and Nicolas Tabareau. 2023. Impredicative Observational Equality. *Proc. ACM Program. Lang.* 7, POPL (2023), 74:1–74:26.
- [28] Loïc Pujet and Nicolas Tabareau. 2022. Observational Equality: Now for Good. *Proc. ACM Program. Lang. (POPL)* 6, (2022).
- [29] Pierre-Marie Pédro and Nicolas Tabareau. 2020. The Fire Triangle: How to Mix Substitution, Dependent Elimination, and Effects. *Proc. ACM Program. Lang. (POPL)* 4, (2020), 58:1–58:28.
- [30] Sebastian Ullrich and Leonardo de Moura. 2019. Counting Immutable Beans: Reference Counting Optimized for Purely Functional Programming. In *IFL*, 2019.
- [31] Hongwei Xi. 2003. Applied Type System (extended abstract). In *TYPES*, 2003.

A Formal definition

This appendix defines Ochr completely: its syntax and runtime structures (Section A.1), the machine (Section A.2), observation, Id and conversion (Section A.3), typing (Section A.4) and well-formed environments (Section A.5). Every derivation in the paper can be checked against it. Each group of rules names, in small type, the functions of the accompanying Lean checker (Section 8) that implement it (in `Machine.lean` unless noted). Section A.6 records the points where a choice had to be made, and, for each side condition of Figure 7, a counterexample showing why the weaker reading fails.

A.1 Syntax and runtime structures

A.1.1 Terms

$t, u, A, B ::=$	$x \mid \text{Prop} \mid \text{Type}_i$	variable, sorts ($i \geq 0$)
	$\mid \Pi(x_1 : A_1 \dots x_n : A_n).B$	dependent function type ($n \geq 1$)
	$\mid \text{fix } f(x_1 : A_1 \dots x_n : A_n) : B \text{ by } x_j := t$	function, recursive on x_j
	$\mid \text{fix } f(x_1 : A_1 \dots x_n : A_n) : B := t$	function, not recursive (λ)
	$\mid t(u_1, \dots, u_n)$	saturated call
	$\mid D(a_1, \dots, a_l) \mid C(a_1, \dots, a_l; t_1, \dots, t_k)$	inductive type, constructor
	$\mid \text{Eq } A \ t \ u \mid J(A, a, b, P, h, t)$	equality (primitive), transport
	$\mid \&A \mid p \mid \&p \mid p := t$	borrow type; read, borrow, assign
	$\mid \text{let } x = t; u \mid \text{let } x : A = t; u \mid t; u$	sequencing
	$\mid \text{match } p \{ C_1(\bar{y}_1) \Rightarrow t_1, \dots, C_m(\bar{y}_m) \Rightarrow t_m \}$	case analysis, one arm per constructor ($m \geq 0$)
	$\mid \text{ld } A \ t \ u$	equality of computations
	$\mid v$	embedded value
$p, q ::=$	$x \mid *p \mid p.g$	places (g a field name)
$\mathcal{P} ::=$	$d \mid (f := \text{fix } \dots) \mid \mathcal{P} \ \mathcal{P}$	programs
$d ::=$	$\text{inductive } D(x_1 : X_1 \dots x_l : X_l) : s := C_1(\overline{g_1 : A_1}) \mid \dots \mid C_m(\overline{g_m : A_m})$	declaration

Figure 10: Terms and programs. Embedded values occur only in sealed programs and in the code of closures, never in source programs.

Inductive types. A declaration introduces a type former D with uniform parameters $x_1 : X_1, \dots, x_l : X_l$, a sort $s \in \{\text{Type}_0, \text{Prop}\}$, and $m \geq 0$ constructors C_i , each with named fields $g_{i1} : A_{i1}, \dots, g_{ik_i} : A_{ik_i}$, whose types may mention the parameters. Every constructor builds a value of $D(\bar{x})$; there are no indices. At parameters $\bar{a}$ the type is $D(\bar{a})$, written D when $l = 0$, and the field types are $A_{ij}[\bar{a}/\bar{x}]$. A constructor takes the parameters as leading arguments, $C(\bar{a}; \bar{t})$, and a constructor value records them; we omit them in examples when they can be inferred from the fields or an annotation (`Intro(h, k)` for `Intro(P, Q; h, k)`), and when there are none (`S t`). The library declares

```

inductive Nat : Type0 := Z | S(pred : Nat)      inductive Unit : Type0 := Tt      inductive False : Prop
inductive Pair (A : Type0)(B : Type0) : Type0 := Mk(fst : A, snd : B)
inductive True : Prop := I                      inductive And (P : Prop)(Q : Prop) : Prop := Intro(l : P, r : Q)

```

`S t` abbreviates `S(t)`; `()`, `A × B`, `(a, b)`, `⊤`, `refl`, `P ∧ Q` and `(h, k)` are notation for `Tt`, `Pair(A, B)`, `Mk(a, b)`, `True`, `I`, `And(P, Q)` and `Intro(h, k)`; and `t.1`, `t.2` are the first and second fields, so `p.1` is the predecessor place of a number. A pair is taken apart like any value, by a `match` or through its field places; there is no η rule. `False`, `True` and `And` are known to conversion, because the computation rules of `Eq` produce them (Section A.3); nothing else distinguishes them from other declarations.

Binding. In $\Pi(\bar{x}:\bar{A}). B$ and `fix f ( $\bar{x}:\bar{A}$ ) : B ... := t`, each x_i is bound in $A_{i+1} \dots A_n$, in B and in t . With `by xj, f` is bound in t and nowhere else; without `by`, f is not bound at all, and in no case is f bound in $\bar{A}$ or B . `let`

$x = t$; u binds x in u . In a declaration, each parameter x_i is bound in $X_{i+1}, \dots, X_l$ and in the field types. Terms are identified up to renaming of bound variables.

Pattern variables are places. The variables $\bar{y}_i = y_1, \dots, y_{k_i}$ of the arm for C_i are not binders: we identify the arm t_i with $t_i[p.g_{i1}/y_1, \dots, p.g_{ik_i}/y_{k_i}]$, which replaces the root y_j of every place in t_i by the field place $p.g_{ij}$ (for Nat , y becomes $p.1$ and $y.1$ becomes $p.1.1$), and write the resolved match $\text{match } p\{C_1 \Rightarrow t_1, \dots, C_m \Rightarrow t_m\}$, abbreviated m below. Every definition below (free places, footprints, captures) is on such resolved terms.

Borrow types. $\&A$ is well formed only when A is a *data type*: its value, once evaluated, is headed by an inductive declared in Type_0 (at any parameters, so $\&\text{Box}(\text{Prop})$ is allowed, and pairs are data); never a sort, a Π -type or a proposition ($[\text{T-Ref}]$). A type variable or a stuck type is not known to be data and is rejected; this fails safe, since a less refined path rejects rather than deciding differently, and it costs any generic $\&A$ for $A : \text{Type}_0$ (note 11). And $\&$ occurs only syntactically at the top of a declared type, as the whole declared type of a variable, a parameter or a function's result: never inside another type, and never as the value of a type term that is not written $\&A$ (a codomain that computes to $\&\text{Nat}$ is an error), so that $[\text{Close}]$'s row, read from the declared codomain, always sees a borrow result. There are no borrows inside data and no borrows of borrows. The machine also checks this where values are built ($[\text{Pair}]$, $[\text{Borrow}]$, closure capture).

checker: `isDataType, Term.refs0k, refTop0k` (`Basic.lean`)

Type positions. The following positions of a term are *type positions*: the annotation A of $\text{let } x : A = t$; u ; the parameter types and the codomain of Π and of fix ; every argument of $\&$ and Eq , and the parameters of an inductive type $D(\bar{a})$; the first argument of Id (whose two sides run on private copies anyway, Section A.3); and the arguments A , a , b and P of J . What stands in a type position is erased (Section A.4.1).

A.1.2 Values, neutrals and types

$v, w, \kappa ::= C(\bar{v}; v_1, \dots, v_k)$	data ($Z, S, v, ()$), (v, w), ...)
$\quad \mid \star$	the value of every proof
$\quad \mid \text{borrow}_\ell v \mid \text{loan}_\ell \mid \perp$	borrow, loan, moved-out place
$\quad \mid n \mid F \mid T$	neutrals, function values, types
$n ::= \sigma \mid [t]$	abstract value, sealed program
$F ::= f \mid \langle \bar{\kappa} \vdash \text{fix } f(\bar{x} : \bar{A}) : B \dots := t \rangle$	top-level function, closure
$T ::= s \mid D(v_1, \dots, v_l) \mid \&T \mid \text{Eq } T \ v \ w$	types (Nat , False , $\text{And}(T, T')$, ...)
$\quad \mid \langle \bar{\kappa} \vdash \Pi(\bar{x} : \bar{A}).B \rangle \mid n$	Π -closure, neutral type
$s ::= \text{Prop} \mid \text{Type}_i$	sorts
$\Omega ::= \varphi_1; \dots; \varphi_k$	frames, oldest first
$\varphi ::= (x : A \mapsto v)^* v^*$	bindings, then temporaries

Figure 11: Values and environments. ℓ ranges over loan labels and σ over abstract values.

A *closure* $\langle \bar{\kappa} \vdash t \rangle$ is code t whose free variables are bound, in order, to the captured values $\bar{\kappa}$; a *Π -closure* is the same for a Π -type. They are the paper's "closures" and " Π -types are closures over values". A top-level function f is a value by itself; its code is in the signature Σ . A neutral is a type exactly when its type is a sort; for instance the sealed program $[\text{Le}(\sigma, \sigma')]$ is a proposition.

A *sealed program* $[t]$ contains a closed term t : it has no free variables, though it may embed values, including abstract values and loans. $[\text{Close}]$ produces sealed programs of the form $[L; C; K]$ with a distinguished *head call* C , which we mark $f(\bar{a})^h$.

Loans. $\text{loans}(v)$ is the set of labels ℓ such that loan_ℓ occurs in v , looking inside sealed programs, closures and types. In an environment Ω , ℓ is *live* if borrow_ℓ occurs in Ω , and *inert* otherwise; inert loans occur only inside the runs of $[\text{Seal}]$. A value is *loan-free* if it contains no live loan, and *borrow-free* if moreover it contains no borrow_ℓ .

A.1.3 Environments and states

An environment $\Omega = \varphi_1; \dots; \varphi_k$ is a stack of frames, oldest first. A frame holds a sequence of *bindings* $x : A \mapsto v$, where the stored type A is omitted for the parameters bound by the untyped machine, followed by a sequence of *temporaries* v , the values in flight during evaluation (evaluated arguments, the right-hand side of an assignment,

a result while its frame is popped). The *positions* π of Ω are its bindings and temporaries, ordered by frame, then bindings before temporaries, then by index; “in the order of Ω ” refers to this order. Variables are resolved in the top frame. We write $\Omega(\pi)$ and $\Omega(x)$ for contents, $\Omega + (x : A \mapsto v)$ for a new binding in the top frame, $\Omega \cdot v$ for a new temporary, and $\Omega; \varphi$ for a pushed frame.

A *state* is an environment together with: Δ , the type of every abstract value; ρ , the refinements made so far, of abstract values and of generalised sealed programs; the goal G while a definition is being checked; and the [Rec] stack. We write Ω for the whole state and mention the other components only where a rule changes them. Δ and the generalisation records of ρ are *global*: a private copy discards its changes to the environment, the goal and the refinements of abstract values, but keeps every abstract value it introduced and every generalisation it recorded, and fresh names (abstract values and loan labels) are never reused. The global signature Σ records every inductive declaration, and for each top-level definition its Π -type and its code. A program is checked item by item ([Ind], [Def], [Const]) against the signature of the items before it.

checker: `Env.lean`, `Check.lean`

A.1.4 Auxiliary definitions

1. **Content and update.** $\text{content}_\Omega(x) = \Omega(x)$; $\text{content}_\Omega(*p) = w$ if $\text{content}_\Omega(p) = \text{borrow}_\ell w$; $\text{content}_\Omega(p.g) = v_j$ if $\text{content}_\Omega(p) = \mathbf{C}(\bar{a}; v_1, \dots, v_k)$ and g is the j -th field of $\mathbf{C}$, and $\text{content}_\Omega(p.g) = \star$ if $\text{content}_\Omega(p) = \star$ (a field of a proof is a proof, Section A.4.6); otherwise $\text{content}_\Omega(p)$ is undefined (so reading $\mathbf{p}.g$ needs a known constructor with field g , as reading $\mathbf{p}.1$ needs a known $\mathbf{S}$, and $*\mathbf{p}$ needs a borrow). $\Omega[p \mapsto v]$ replaces that sub-value. The *prefixes* of a place are $\text{pre}(x) = \{x\}$, $\text{pre}(*p) = \text{pre}(p) \cup \{*p\}$, $\text{pre}(p.g) = \text{pre}(p) \cup \{p.g\}$; $q \sqsubseteq p$ means $q \in \text{pre}(p)$.

checker: `content`, `setPlace`

2. **Loans an access must end** ([Access]). For reading, borrowing and assigning,

$$L_\Omega^R(p) = \{\ell \mid \text{content}_\Omega(q) = \text{loan}_\ell, q \in \text{pre}(p)\} \cup \text{loans}(\text{content}_\Omega(p)),$$

and for matching, $L_\Omega^M(p) = \{\ell \mid \text{content}_\Omega(q) = \text{loan}_\ell, q \in \text{pre}(p)\} \cup \text{hl}(\text{content}_\Omega(p))$, where $\text{hl}([t]) = \text{loans}(t)$ (a loan’s position inside a neutral is unknown, so it is treated as the head) and $\text{hl}(v) = \emptyset$ for every other v . Both keep only live labels, ordered from the root of $\mathbf{p}$ outward and then left to right inside the content.

checker: `accessPath`, `accessInside`

3. **Owners.** For a live label ℓ ,

$$\text{owners}_\Omega(\ell) = \bigcup_{\pi: \ell \in \text{loans}(\Omega(\pi))} \begin{cases} \text{owners}_\Omega(m) & \text{if } \Omega(\pi) = \text{borrow}_m w \\ \{\pi\} & \text{otherwise,} \end{cases}$$

a set of positions of Ω (well-defined by acyclicity, Section A.5). The owners of a variable are $\text{own}_\Omega(x) = \text{owners}_\Omega(\ell)$ if $\Omega(x) = \text{borrow}_\ell w$, $\emptyset$ if $\Omega(x) = \perp$, and $\{x\}$ otherwise.

checker: `owners` (`Obs.lean`)

4. **Place occurrences.** $\text{occ}(t)$ is the set of free place occurrences of a resolved term $\mathbf{t}$, each with its kind: read (rd), borrowed (bw), assigned (as) or matched (sc). $\text{occ}(p) = \{(p, \text{rd})\}$, $\text{occ}(\&p) = \{(p, \text{bw})\}$, $\text{occ}(p := t) = \{(p, \text{as})\} \cup \text{occ}(t)$, $\text{occ}(\text{match } p\{\mathbf{C}_1 \Rightarrow t_1, \dots, \mathbf{C}_m \Rightarrow t_m\}) = \{(p, \text{sc})\} \cup \bigcup_i \text{occ}(t_i)$; a binder removes the places rooted at the variable it binds; every other form takes the union over its subterms, including the types and the code of Π and `fix`. A variable x is a *borrow variable* in Ω if its stored type is $\&\mathbf{T}$ or $\Omega(x)$ is a borrow.

checker: `Term.place0ccs`, `Term.free0ccs` (`Basic.lean`)

5. **Footprint.**

$$W_\Omega(t, u) = \bigcup \{\text{own}_\Omega(\text{root}(p)) \mid (p, k) \in \text{occ}(t) \cup \text{occ}(u), k \in \{\text{bw}, \text{as}\} \text{ or } \text{root}(p) \text{ is a borrow variable}\},$$

ordered as Ω . A place rooted at an owned variable that is only read contributes nothing.

checker: `footprint` (`Obs.lean`)

6. **Substitution and normal form.** For an atom $a \in \{\sigma, \text{loan}_\ell\}$, or a neutral n (for generalisation), $v[w/a]$ replaces every occurrence of a in v , including inside sealed programs, closures and types, and then restores normal form: every sealed program that changed is replaced by its normal form ([Seal]), and every **Eq** type is rebuilt by the smart constructor eq of Section A.3. $\Omega[w/a]$ does this to every value of Ω ; $\Omega[w/a]^+$ also to the stored types, to Δ and to the goal.

checker: `substV, substT, substEnv`

7. **Refinement and generalisation** ([Split]). A refinement of an abstract value σ of inductive type $D(\bar{a})$, with D declared in **Type₀**, is $\sigma := C(\bar{a}; \sigma_1, \dots, \sigma_k)$ for a constructor $C(g_1 : A_1, \dots, g_k : A_k)$ of D , with σ_j fresh and $\Delta(\sigma_j) = A_j[\bar{a}/\bar{x}]$ (proofs are never refined: their value is $\star$); $\Omega[\sigma := r]$ is $\Omega[r/\sigma]^+$, and ρ records it. *Generalising* a neutral $n = [t]$ (or an inert loan) at a place of type T takes σ fresh with $\Delta(\sigma) = T$, forms $\Omega[\sigma/n]^+$, and records $n := \sigma$ in ρ , so that it persists: whenever normalisation derives n again it yields $\rho^*(\sigma)$ ([Seal-stuck]), and whenever ρ grows every sealed program of the state is re-normalised. The *refined value* $\rho^*(v)$ applies ρ repeatedly, through constructors; the *strict subterms* are $\text{sub}(C(v_1, \dots, v_k)) = \bigcup_j (\{v_j\} \cup \text{sub}(v_j))$ and $\text{sub}(v) = \emptyset$ otherwise.

checker: `refine, expandRefs, strictSubterms, generalizeNeutral, canonNeutral, renormAll`

8. **Drop.** $\text{drop}(\Omega, v) = \Omega'$ where $\Omega \cdot v \xrightarrow{\ell} \Omega' \cdot \perp$ ([End]) if $v = \text{borrow}_\ell w$; $\text{drop}(\Omega, v) = \Omega$ if v is loan-free; otherwise $\text{drop}(\Omega, v) = \text{err}$ (something still borrows a dying value). Popping a frame drops its bindings, newest first, with the call's result held as a temporary of the caller's frame: `pop`($\Omega; \varphi, v$).

checker: `dropTopBind, dropValue, popFrame`

9. **Types of places and values.** $\text{type}_\Omega(x)$ is the stored type of x , or $\text{typeof}(\Omega(x))$ if it has none; $\text{type}_\Omega(*p) = T$ if $\text{type}_\Omega(p) = \&T$; $\text{type}_\Omega(p.g) = A_j[\bar{a}/\bar{x}]$ if $\text{type}_\Omega(p) = D(\bar{a})$ with $g_j : A_j$ the field g of a constructor C of D , and either $\text{content}_\Omega(p) = C(\dots)$ or $\text{content}_\Omega(p) = \star$ and C is the only constructor of D (so a field's type is known once its constructor is, and a proof's constructor is known from its type). For values: $C(\bar{a}; \bar{v})$ has type $D(\bar{a})$ for C a constructor of D ; (v, w) has $\text{typeof}(v) \times \text{typeof}(w)$; σ has $\Delta(\sigma)$; f has its type in Σ ; a closure $\langle \bar{\kappa} \vdash \text{fix } f(\bar{x} : \bar{A}) : B \dots \rangle$ has the Π -closure $\langle \bar{\kappa} \vdash \Pi(\bar{x} : \bar{A}).B \rangle$; $\text{borrow}_\ell w$ has $\&\text{typeof}(w)$; a type has its sort (Section A.4).

checker: `placeType, valType`

10. **The generic call.** For a Π -closure $\Pi = \langle \bar{\kappa} \vdash \Pi(x_1 : A_1 \dots x_n : A_n).B \rangle$, the *generic arguments* $\bar{a}$ are built left to right. In the frame $\bar{\kappa}, x_1 : T_1 \mapsto a_1, \dots, x_{i-1} : T_{i-1} \mapsto a_{i-1}$, evaluate A_i in a type position to T_i (Section A.4). If $T_i = \&T$, add an owned place $c_i : T \mapsto \text{loan}_{\ell_i}$ to a bottom frame and let $a_i = \text{borrow}_{\ell_i} \sigma_i$ with $\sigma_i : T$ and ℓ_i fresh (this is the argument $\&c_i$ after [Borrow]); if T_i has sort **Prop**, let $a_i = \star$; otherwise $a_i = \sigma_i$ with $\sigma_i : T_i$ fresh. The *generic state* is the frame of owned places $\Gamma(\Pi) = (c_i : T \mapsto \text{loan}_{\ell_i})_{i: T_i = \&T}$, and the *parameter frame* is $\varphi(\Pi) = (\bar{\kappa}, \bar{x} : \bar{T} \mapsto \bar{a})$, each type stored with its binding. The generic call is $F(\bar{a})$ at $\Gamma(\Pi)$.

checker: `checkFix`

A.2 The machine

The machine is the big-step judgement $\langle \Omega, t \rangle \Downarrow r$, where the outcome r is $\langle \Omega', v \rangle$, **stuck** (the run needs to inspect a neutral) or **err** (a borrow error). It is deterministic. **Propagation:** when a premise that evaluates a subterm has outcome **stuck** or **err**, so does the conclusion; the only rules that inspect a **stuck** premise are [App-close], [App-head] and [Seal]. We write $\langle \Omega, t \rangle \Downarrow \langle \Omega', v \rangle$ only for successful runs.

A.2.1 Borrows

Ending a borrow replaces it by $\perp$ and substitutes its content for its loan everywhere, in normal form (Section A.1.4, item 6). A borrow occurs only as the whole value of a binding or temporary (Section A.5), so it has a position π .

$$\frac{\Omega(\pi) = \text{borrow}_{\ell} w}{\Omega \xrightarrow{\ell} (\Omega[\pi \mapsto \perp])[w/\text{loan}_{\ell}]} \text{END } \ell$$

[**Access**]. $\text{acc}_p^R(\Omega)$ and $\text{acc}_p^M(\Omega)$ end, one at a time and in the order of item 2 of Section A.1.4, the borrow of the first label of $L_\Omega^R(p)$, respectively $L_\Omega^M(p)$, until that set is empty:

$$\text{acc}_p^X(\Omega) = \begin{cases} \Omega & \text{if } L_\Omega^X(p) = \emptyset \\ \text{acc}_p^X(\Omega') & \text{if } \ell \text{ is the first label of } L_\Omega^X(p) \text{ and } \Omega \xrightarrow[\ell]{\sim} \Omega' \end{cases} \quad (X \in \{R, M\}).$$

Every rule that reads, borrows or assigns a place **p** first computes acc_p^R , and a match on **p** computes acc_p^M . This is the borrow checker: an ended borrower holds $\perp$, and every later use of it is an error.

checker: `endBorrow`, `accessPath`, `accessInside`, `accessNeutralHead`

$$\begin{array}{c} \frac{\text{acc}_p^R(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = v \notin \{\perp, \text{borrow}_\ell w\}}{\langle \Omega, p \rangle \Downarrow \langle \Omega_1, v \rangle} \text{ READ} \\[10pt] \frac{\text{acc}_p^R(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = \text{borrow}_\ell w}{\langle \Omega, p \rangle \Downarrow \langle \Omega_1[p \mapsto \perp], \text{borrow}_\ell w \rangle} \text{ MOVE} \quad \frac{\text{acc}_p^R(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) \text{ undefined or } \perp}{\langle \Omega, p \rangle \Downarrow \text{err}} \text{ READ-ERR} \\[10pt] \frac{\text{acc}_p^R(\Omega) = \Omega_1 \quad \ell \text{ fresh} \quad \text{content}_{\Omega_1}(p) = v \notin \{\perp, \text{borrow}_m w\}}{\langle \Omega, \&p \rangle \Downarrow \langle \Omega_1[p \mapsto \text{loan}_\ell], \text{borrow}_\ell w \rangle} \text{ BORROW} \\[10pt] \frac{\text{acc}_p^R(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) \text{ undefined, } \perp \text{ or a borrow}}{\langle \Omega, \&p \rangle \Downarrow \text{err}} \text{ BORROW-ERR} \\[10pt] \frac{\langle \Omega, t \rangle \Downarrow \langle \Omega_1, v \rangle \quad \text{acc}_p^R(\Omega_1 \cdot v) = \Omega_2 \cdot v' \quad \text{content}_{\Omega_2}(p) = w \quad \text{drop}(\Omega_2, w) = \Omega_3}{\langle \Omega, p := t \rangle \Downarrow \langle \Omega_3[p \mapsto v'], () \rangle} \text{ ASSIGN} \end{array}$$

In [Assign] the new value travels as a temporary while **p** is accessed, so that ending a borrow can substitute into it; `drop`(Ω_2, w) ends the old content if it is a borrow, and fails if it still holds a live loan.

checker: `readPlace`, `borrowPlace`, `assignPlace`

A.2.2 Sequencing and data

$$\begin{array}{c} \frac{\langle \Omega, t \rangle \Downarrow \langle \Omega_1, v \rangle \quad \langle \Omega_1 + (x \mapsto v), u \rangle \Downarrow \langle \Omega_2 + (x \mapsto v'), w \rangle \quad \text{drop}(\Omega_2 \cdot w, v') = \Omega_3 \cdot w'}{\langle \Omega, \text{let } x = t; u \rangle \Downarrow \langle \Omega_3, w' \rangle} \text{ LET} \\[10pt] \frac{\langle \Omega, t \rangle \Downarrow \langle \Omega_1, v \rangle \quad \text{drop}(\Omega_1, v) = \Omega_2 \quad \langle \Omega_2, u \rangle \Downarrow r}{\langle \Omega, t; u \rangle \Downarrow r} \text{ SEQ} \end{array}$$

The annotated `let x : A = t; u` runs as `let x = t; u`: its annotation is a type position. A discarded value (`t; u`) is dropped like a binding: a borrow ends, and a live loan is an error.

checker: `eval (.letIn, .seq)`, `dropTopBind`, `dropValue`

$$\frac{\langle \Omega, \bar{t} \rangle \Downarrow^* \langle \Omega', \bar{v} \rangle}{\langle \Omega, \mathbf{C}(\bar{t}) \rangle \Downarrow \langle \Omega', \mathbf{C}(\bar{v}) \rangle} \text{ CTOR}$$

A constructor evaluates its fields left to right, each into a temporary ([Args], below); its field types are borrow-free, so a constructor value holds no borrow. A pair is the constructor **Mk**.

checker: `evalCore (.ctor, .zero, .succ, .tt, .fst, .snd)`

A.2.3 Functions, types and proofs

A closure or Π -type captures, when it is formed, the current contents of its free variables $y_1, \dots, y_m$ (in the order of Ω), each accessed as a read: $\Omega_0 = \Omega$, $\Omega_i = \text{acc}_{y_i}^R(\Omega_{i-1})$, $\kappa_i = \text{content}_{\Omega_i}(y_i)$, each recorded with the declared type and proof flag of y_i 's binding (note 27). A captured value may be neither $\perp$ nor a borrow (closures capture no borrows); otherwise the rule fails. The other type formers compute type values; **Eq** uses the smart constructor of Section A.3.

$$\begin{array}{c}
\frac{f \in \Sigma}{\langle \Omega, f \rangle \Downarrow \langle \Omega, f \rangle} \text{GLOBAL} \quad \frac{\bar{\kappa} \text{ captured}, \Omega_m}{\langle \Omega, \mathbf{fix} \ f(\bar{x} : \bar{A}) : B \dots := t \rangle \Downarrow \langle \Omega_m, \langle \bar{\kappa} \vdash \mathbf{fix} \ f(\bar{x} : \bar{A}) : B \dots := t \rangle \rangle} \text{FIX} \\
\\
\frac{\bar{\kappa} \text{ captured}, \Omega_m}{\langle \Omega, \Pi(\bar{x} : \bar{A}).B \rangle \Downarrow \langle \Omega_m, \langle \bar{\kappa} \vdash \Pi(\bar{x} : \bar{A}).B \rangle \rangle} \text{PI} \quad \frac{}{\langle \Omega, s \rangle \Downarrow \langle \Omega, s \rangle} \text{SORT} \quad \frac{\langle \Omega, \bar{a} \rangle \Downarrow^* \langle \Omega', \bar{v} \rangle}{\langle \Omega, D(\bar{a}) \rangle \Downarrow \langle \Omega', D(\bar{v}) \rangle} \text{IND} \\
\\
\frac{\langle \Omega, A \rangle \Downarrow \langle \Omega', T \rangle}{\langle \Omega, \&A \rangle \Downarrow \langle \Omega', \&T \rangle} \text{REF} \quad \frac{\langle \Omega, A \rangle \Downarrow \langle \Omega_1, T \rangle \quad \langle \Omega_1, a \rangle \Downarrow \langle \Omega_2, v \rangle \quad \langle \Omega_2, b \rangle \Downarrow \langle \Omega_3, w \rangle}{\langle \Omega, \mathbf{Eq} \ A \ a \ b \rangle \Downarrow \langle \Omega_3, \text{eq}(T, v, w) \rangle} \text{EQ} \\
\\
\frac{a \text{ and } b \text{ evaluate to convertible values} \quad \langle \Omega, t \rangle \Downarrow r}{\langle \Omega, \mathbf{J}(A, a, b, P, h, t) \rangle \Downarrow r} \text{J} \\
\\
\frac{a \text{ and } b \text{ evaluate to values that are not convertible}}{\langle \Omega, \mathbf{J}(A, a, b, P, h, t) \rangle \Downarrow \mathbf{stuck}} \text{J-STUCK}
\end{array}$$

In [Ind] the parameters are evaluated like arguments ([Args], below); a type **And**(T, T') is kept as written, and the unit laws apply to it only in conversion ([Conv-unit]). **Id A t u** is a type former too; its value is given in Section A.3. A constructor of an inductive declared in **Prop** (such as **refl** or **(h, k)**) is a declared proof, so the machine never evaluates it ([Erase-proof]). **J** returns the value of **t** when its endpoints are convertible, as Lean's **Eq.rec** does, and is stuck otherwise, so a cast under a hypothesis that equates unequal values never produces a value of the wrong type. **a** and **b** are evaluated in type positions, on private copies; **A**, **P** and the proof **h** are never run by the machine. A top-level constant **c : A := t** evaluates to the value stored in Σ , which is $\star$ when **A** has declared sort **Prop**.

checker: capture, evalCore (.pi, .fix, .sort, .prod, .ref, .eq, .tind, .prim "J", .const), evalTInd, ctorIsProof

Erased terms. Whether an occurrence is erased is read from syntax and declarations, never from a normal form or a computed type (Section A.4.1); the machine needs no types for it, only one flag per binding, saying whether the binding was declared a proof. An erased term runs on a private copy of the environment: its effects are discarded and only its value is kept, which for a proof is $\star$, so a proof need not be run at all.

$$\begin{array}{c}
\frac{t \text{ erased, a declared proof}}{\langle \Omega, t \rangle \Downarrow \langle \Omega, \star \rangle} \text{ERASE-PROOF} \\
\\
\frac{t \text{ erased, a type} \quad \langle \Omega, t \rangle \Downarrow_0 \langle \Omega', T \rangle \text{ confined}}{\langle \Omega, t \rangle \Downarrow \langle \Omega, T \rangle} \text{ERASE-TYPE} \quad \frac{t \text{ erased} \quad \langle \Omega, t \rangle \Downarrow_0 \langle \Omega', v \rangle \text{ not confined}}{\langle \Omega, t \rangle \Downarrow \mathbf{err}} \text{ERASE-ERR}
\end{array}$$

Here $\Downarrow_0$ is the judgement in which the rule for **t**'s own form is applied at the root instead of [Erase-type]. A run of **t** from Ω is *confined* if none of its steps assigns, borrows or moves out of a place rooted at a position of Ω , except a borrow or move that evaluates an argument of an erased call; a step inside nested erased occurrences is judged by the outermost erased occurrence containing it, so an erased term may mutate its own locals freely. A proof that the machine skips ([Erase-proof]) was checked to be confined by [T-Erase] when its enclosing definition was checked.

checker: eval, callFn, fnClass, paramFlags, onCopy (Env.lean)

A.2.4 Calls

Arguments are evaluated left to right, each into a temporary of the caller's top frame, so that [Access] can see, and end, the earlier ones:

$$\begin{array}{c}
\frac{}{\langle \Omega, \varepsilon \rangle \Downarrow^* \langle \Omega, \varepsilon \rangle} \text{ARGS-NIL} \quad \frac{\langle \Omega, u \rangle \Downarrow \langle \Omega_1, w \rangle \quad \langle \Omega_1 \cdot w, \bar{u} \rangle \Downarrow^* \langle \Omega_2 \cdot w', \bar{w} \rangle}{\langle \Omega, u \bar{u} \rangle \Downarrow^* \langle \Omega_2, w' \bar{w} \rangle} \text{ARGS} \\
\\
\frac{\langle \Omega, t_0 \rangle \Downarrow \langle \Omega_0, F \rangle \quad \langle \Omega_0 \cdot F, \bar{u} \rangle \Downarrow^* \langle \Omega_1 \cdot F', \bar{w} \rangle \quad \text{no } w_i \text{ is } \perp \quad \langle \Omega_1, F'(\bar{w}) \rangle \Downarrow_{\text{app}} r}{\langle \Omega, t_0(\bar{u}) \rangle \Downarrow r} \text{CALL} \quad \frac{\langle \Omega_0 \cdot F, \bar{u} \rangle \Downarrow^* \langle \Omega_1 \cdot F', \bar{w} \rangle \quad \text{some } w_i = \perp}{\langle \Omega, t_0(\bar{u}) \rangle \Downarrow \text{err}} \text{CALL-ERR}
\end{array}$$

An argument is $\perp$ when a later argument ended its borrow, as in $\mathbf{f}(\&\mathbf{x}, \&\mathbf{x})$: it is not of its parameter's type. The judgement $\langle \Omega, F(\bar{w}) \rangle \Downarrow_{\text{app}} r$ applies a function value at its *call point*, the state after the arguments have been evaluated. Let $\mathbf{F}$ have code $\mathbf{fix} \ f(\bar{x} : \bar{A}) : B \ [\mathbf{by} \ x_j] := b$ and captured values $\bar{\kappa}$ (none for a top-level $\mathbf{f}$), and let φ be the frame $\bar{\kappa}$, $[f \mapsto F]$, $\bar{x} \mapsto \bar{w}$, where $\mathbf{f}$ is bound only with $\mathbf{by}$.

$$\begin{array}{c}
\frac{\langle \Omega; \varphi, b \rangle \Downarrow \langle \Omega'; \varphi', v \rangle \quad \text{pop}(\Omega'; \varphi', v) = \langle \Omega'', v' \rangle}{\langle \Omega, F(\bar{w}) \rangle \Downarrow_{\text{app}} \langle \Omega'', v' \rangle} \text{APP} \\
\\
\frac{\langle \Omega; \varphi, b \rangle \Downarrow \text{stuck} \quad \text{the call is not a head call}}{\langle \Omega, F(\bar{w}) \rangle \Downarrow_{\text{app}} \text{close}(\Omega, F, \bar{w})} \text{APP-CLOSE} \\
\\
\frac{\langle \Omega; \varphi, b \rangle \Downarrow \text{stuck} \quad \text{the call is a head call } F(\bar{w})^h}{\langle \Omega, F(\bar{w})^h \rangle \Downarrow_{\text{app}} \text{stuck}} \text{APP-HEAD} \\
\\
\frac{F = n \text{ a neutral} \quad \text{the call is not a head call}}{\langle \Omega, n(\bar{w}) \rangle \Downarrow_{\text{app}} \text{close}(\Omega, n, \bar{w})} \text{APP-NEUTRAL} \quad \frac{F = n \text{ a neutral}}{\langle \Omega, n(\bar{w})^h \rangle \Downarrow_{\text{app}} \text{stuck}} \text{APP-NEUTRAL-HEAD}
\end{array}$$

[App-close] discards the partial run of the body: close starts again from the call point Ω . The head guard of [Seal] covers neutral-headed calls too: a sealed program whose head call has a neutral head stays as it is.

checker: evalCall, callFn, runBody, popFrame

A.2.5 Closing off

$\text{close}(\Omega, F, \bar{w})$ closes off the call $F(\bar{w})$ at its call point Ω . Its precondition is that every argument is loan-free or is $\text{borrow}_{\ell} u$ with u loan-free; [Access] guarantees it. Let $I = \{i \mid w_i = \text{borrow}_{\ell_i} u_i\}$, let c_i ($i \in I$) be fixed names, and let

$$L := (\text{let } c_i = u_i;)_{i \in I} \quad C := F(a_1, \dots, a_n)^h \quad \text{with } a_i = \begin{cases} \&c_i & \text{if } i \in I \\ w_i & \text{otherwise.} \end{cases}$$

The row is chosen by the *declared* codomain $\mathbf{B}$ of $\mathbf{F}$'s type: the codomain term of $\mathbf{f}$'s definition, of the closure's code, of $\Delta(\sigma)$ for an abstract function, or the type of a stuck block.

declared codomain $\mathbf{B}$	result $\mathbf{r}$	each $\text{loan}_{\ell_i}, i \in I$, becomes f_i
Unit	$()$	$[\mathbf{L}; \mathbf{C}; \ c_i]$
$\&\mathbf{T}$ (k fresh)	$\text{borrow}_k[\mathbf{L}; \text{let } \mathbf{r} = \mathbf{C}; \ * \mathbf{r}]$	$[\mathbf{L}; \text{let } \mathbf{r} = \mathbf{C}; \ * \mathbf{r} := \text{loan}_k; \ c_i]$
any other	$[\mathbf{L}; \mathbf{C}]$	$[\mathbf{L}; \mathbf{C}; \ c_i]$

Figure 12: [Close]: $\text{close}(\Omega, F, \bar{w}) = \langle \Omega[f_i/\text{loan}_{\ell_i}]_{i \in I}, r \rangle$. The borrows borrow_{ℓ_i} are consumed: at the call point they are no longer in Ω .

checker: closeCall, declKind, resultKind, kindOf

A.2.6 Matching

$$\begin{array}{c}
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = C_i(\bar{v}) \quad \langle \Omega_1, t_i \rangle \Downarrow r}{\langle \Omega, \mathbf{match} \, p\{C_1 \Rightarrow t_1, \dots, C_m \Rightarrow t_m\} \rangle \Downarrow r} \text{MATCH} \\
\\
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) \text{ is a neutral or an inert loan}}{\langle \Omega, \mathbf{match} \, p\{\dots\} \rangle \Downarrow \mathbf{stuck}} \text{MATCH-STUCK} \\
\\
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) \text{ is undefined or } \perp}{\langle \Omega, \mathbf{match} \, p\{\dots\} \rangle \Downarrow \mathbf{err}} \text{MATCH-ERR} \\
\\
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad C_1 \text{ is the only constructor of a D declared in Prop} \quad \langle \Omega_1, t_1 \rangle \Downarrow r}{\langle \Omega, \mathbf{match} \, p\{C_1 \Rightarrow t_1\} \rangle \Downarrow r} \text{MATCH-PROP} \\
\\
\frac{\mathbf{match} \, p\{\} \text{ not in a proof position}}{\langle \Omega, \mathbf{match} \, p\{\} \rangle \Downarrow \mathbf{stuck}} \text{MATCH-NONE}
\end{array}$$

In arm i the pattern variables are the field places $p.g_{ij}$, so $\&y$ reborrows a field in place (for **Nat**, the predecessor $p.1$). A match whose arms name the constructors of an inductive declared in **Prop** is a *match on a proof*. The machine recognises it from those names, which is a fact about declarations, and never inspects the content of p , which is $\star$: with one constructor it takes the only arm ([Match-prop]), in which the fields of p hold $\star$ (Section A.1.4, item 1); with several, the match is a declared proof (Section A.4.1) and is never run. A match with no arms in a proof position is erased like any proof; anywhere else, reaching it is stuck ([Match-none]), since it is reached only under a hypothesis that a refinement has made false. [Match], [Match-stuck] and [Match-err] apply to the other matches.

checker: evalMatch, evalMatchInd, accessNeutralHead, byTypeMatch, evalMatchByType

A.2.7 Sealed programs

Values are kept in normal form. The normal form of a sealed program is computed by running it from the empty environment ε (one empty frame), where its head call may not close off ([App-head]):

$$\begin{array}{c}
\frac{\langle \varepsilon, t \rangle \Downarrow \langle \Omega', v \rangle}{\text{nf}(\lceil t \rceil) = v} \text{SEAL} \quad \frac{\langle \varepsilon, t \rangle \Downarrow \mathbf{stuck}}{\text{nf}(\lceil t \rceil) = \begin{cases} \rho^*(\sigma) & \text{if } \rho \text{ records } \lceil t \rceil := \sigma \\ \text{otherwise} \end{cases}} \text{SEAL-STUCK} \quad \frac{\langle \varepsilon, t \rangle \Downarrow \mathbf{err}}{\text{nf}(\lceil t \rceil) = \mathbf{err}} \text{SEAL-ERR}
\end{array}$$

Every loan in t whose borrow lives outside the run is inert in it: [Access] does not end it (it is not live), [Read] copies it (it is loan-free), drop accepts it, and a match on it is stuck, as on an abstract value. An error is a type error at the point that triggered the normalisation. Normalisation happens whenever a sealed program is created or changed: by [Close], by [End] (substituting a loan), and by refinement or generalisation (Section A.1.4, items 6 and 7); a generalised sealed program stays generalised wherever it is derived again. The embedded values of a stuck sealed program are already normal, because substitution normalises inner sealed programs first.

checker: nfSealed, substV, canonNeutral

A.2.8 Stuck blocks

A stuck match that is not the body of a call is closed off as a call of an anonymous function of its free places, captured as Rust captures closure variables. This happens only in the typing judgement ([Split], Section A.4), after the match's arms have been checked, which gives its type **B** and the set **M** of variables that some arm leaves $\perp$ (moves out of).

Let $m = \mathbf{match} \, p\{C_1 \Rightarrow t_1, \dots, C_m \Rightarrow t_m\}$ be stuck in Ω , and give each occurrence $(q, k) \in \text{occ}(m)$ a *mode*: **mv** (moved) if q is a whole variable x and either x is a borrow variable and $k \in \{\text{rd}, \text{as}\}$, or $x \in M$; otherwise **ref** (borrowed) if $k \in \{\text{bw}, \text{as}\}$; otherwise **cp** (copied). The *captures* $q_1, \dots, q_n$ are the maximal places of $\text{occ}(m)$, those with no strict prefix among its places. Each takes the largest mode of the occurrences at or below it ($\text{cp} < \text{ref} <$

mv), and they are ordered by the position of their root, then by length. Capture q_i becomes a parameter z_i with argument a_i :

$$(z_i : U_i, a_i) = \begin{cases} (z_i : \&type_{\Omega}(q_i), \&q_i) & \text{if } q_i \text{ is ref,} \\ (z_i : type_{\Omega}(q_i), q_i) & \text{otherwise (a move if its content is a borrow, a copy if not).} \end{cases}$$

The body m' is $\mathfrak{m}$ with each place $q_i\pi$ renamed to $(*z_i)\pi$ if q_i is **ref**, and to $z_i\pi$ otherwise. Then

$$\text{block}(\Omega, m, B, M) := F_m(a_1, \dots, a_n) \quad \text{where } F_m = \langle \vdash \mathbf{fix} _ (z_1 : U_1 \dots z_n : U_n) : B := m' \rangle,$$

a non-recursive closure with no captured values. Evaluating the block by [Call] gets stuck on the head of $\mathfrak{m}$ inside F_m 's body and closes off by [App-close], with [Close]'s row chosen by **B**.

checker: `closeOffMatch, splitThenClose`

A.3 Observation, **Id** and conversion

Eq computes. Every **Eq** type is built by the smart constructor `eq`, which takes apart two values built by the same constructor and conjoins the equations between their fields with **and**, which drops **True** conjuncts:

$$\begin{aligned} \text{eq}(\text{D}(\bar{a}), \text{C}(\bar{a}; v_1, \dots, v_k), \text{C}(\bar{a}; w_1, \dots, w_k)) &= \text{and}(\text{eq}(B_1, v_1, w_1), \dots, \text{eq}(B_k, v_k, w_k)) \quad \text{fields } B_j \text{ at } \bar{a}; \text{True if } k = 0 \\ \text{eq}(T, v, w) &= \text{True} \quad \text{if } v \equiv w \text{ (and the first clause does not apply)} \\ \text{eq}(T, \text{C}(\bar{v}), \text{C}'(\bar{w})) &= \text{False} \quad \text{if } \text{C} \neq \text{C}' \text{ are constructors of the same inductive} \\ \text{eq}(T, v, w) &= \text{Eq } T \ v \ w \quad \text{otherwise} \\ \text{and}(\text{True}, P) &= P \quad \text{and}(P, \text{True}) = P \quad \text{and}(P, Q) = \text{And}(P, Q) \quad \text{otherwise} \end{aligned}$$

These realise the conversion rules of Figure 5 for **Eq**, and they are why **Id**'s observation computes to the single interesting equation. A type **And(P, Q)** written in a program is not rebuilt by **and**: it keeps its head, so a proof of $\tau \wedge P$ can be taken apart by a match, and the unit laws reach it only through conversion ([Conv-unit]). The first and third clauses apply only when both sides are constructor values (injectivity and disjointness, note 26 of Section A.6); a neutral is never split. **refl**, the constructor of **True**, has type **True**, so by conversion it proves every reflexive equation; an equation between distinct constructors is **False**, eliminated by **match h {}**.

checker: `mkEqM, mkAnd, distinctCtors` (`Basic.lean`)

Observation. Let $W = \pi_1 < \dots < \pi_k$ be positions of Ω . The observation of $\mathfrak{t}$ at Ω on $\mathfrak{W}$ runs $\mathfrak{t}$ on a private copy of Ω , ends every remaining borrow, and reads off the result and the footprint. A borrow-typed result (only [Conv-fun] observes one, since **Id** needs a borrow-free type) is observed as its current content, after a given abstract value σ_w is written through it, so that its owners show where it points:

$$\begin{aligned} &\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad v \text{ is not a borrow} \quad \Omega_1 \cdot v \xrightarrow{*} \Omega_2 \cdot v' \text{ with no borrow left in } \Omega_2 \cdot v' \quad \text{OBS}}{\llbracket t \rrbracket_{\Omega}^W = \text{tuple}(v', \Omega_2(\pi_1), \dots, \Omega_2(\pi_k))} \\ &\frac{\Omega \vdash t \Downarrow \text{borrow}_k u : \&T \dashv \Omega_1 \quad \Omega_1 \cdot u \cdot \text{borrow}_k \sigma_w \xrightarrow{*} \Omega_2 \cdot u' \cdot \perp \text{ with no borrow left} \quad \text{OBS-BORROW}}{\llbracket t \rrbracket_{\Omega}^{W; \sigma_w} = \text{tuple}(u', \Omega_2(\pi_1), \dots, \Omega_2(\pi_k))} \end{aligned}$$

where $\xrightarrow{*}$ repeatedly ends the first borrow in the order of Ω (so in [Obs-borrow] σ_w reaches the owners of loan_k , as $*r := \sigma_w$ would), $\text{tuple}(v, w_1, \dots, w_k)$ is a tuple of the machine, not a pair value. The run of $\mathfrak{t}$ pops every frame it pushes, so the positions of $\mathfrak{W}$ are still positions of Ω_2 . A stuck match in $\mathfrak{t}$ is closed off by [Split] (Section A.4), so an observation exists unless $\mathfrak{t}$ has an error.

checker: `observe, endAll, tupleVal` (`Obs.lean`)

Id computes. Both sides run from the same Ω , on independent copies:

$$\frac{\Omega \vdash A \Downarrow T \text{ type, } T \text{ contains no } \& \quad W = W_\Omega(t, u) = \pi_1 < \dots < \pi_k \quad \llbracket t \rrbracket_\Omega^W = \text{tuple}(r, w_1, \dots, w_k) \text{ with } t : A_t \equiv T \quad \llbracket u \rrbracket_\Omega^W = \text{tuple}(r', w'_1, \dots, w'_k) \text{ with } u : A_u \equiv T \quad T_i = \text{type}_\Omega(\pi_i) \text{ Id}}{\Omega \vdash \text{Id } A t u \Downarrow \text{and}(\text{eq}(T, r, r'), \text{eq}(T_1, w_1, w'_1), \dots, \text{eq}(T_k, w_k, w'_k)) : \text{Prop} \dashv \Omega}$$

When W is empty, Id is the one equation between the results. A may be any borrow-free type, a proposition or a universe included, since no pair value is built.

checker: `idType`, `footprint`

Conversion. Definitional equality is equality of normal forms, and values are always kept in normal form, so conversion $v \equiv w$ is the least relation closed under the following rules. It is the relation of every “ $\equiv$ ” premise in Section A.4 and of the reflexive clause of `eq`.

- [Conv-refl] $v \equiv v$, where values are identified up to renaming of bound variables (binder names are ignored).
- [Conv-cong] Two values with the same outermost former (a constructor, a borrow, an inductive type, $\&$, `Eq`, a sealed program, a closure) are convertible if their immediate components are pairwise convertible: terms (the code of closures, the programs inside sealed programs) component by component, with embedded values compared by $\equiv$.
- [Conv-unit] $\text{And}(\text{True}, P) \equiv P \equiv \text{And}(P, \text{True})$: two types are convertible if they are after the unit laws are applied to each, through nested `And`s. This is the only way the unit laws act on a type written in a program; stored types keep their `And`.
- [Conv-pi] Two Π -closures with the same number of parameters are convertible if, at the generic state Γ and generic arguments $\bar{a}$ of the first (Section A.1.4, item 10), their parameter types, each evaluated with the earlier parameters bound to $\bar{a}$, are pairwise convertible, and so are their codomains. So Π -types are compared under binders by instantiating the binders, as function values are ([Conv-fun]), and captured values matter only through the types they help compute: $\langle m \mapsto \sigma \vdash \Pi(n : \text{Nat}). \text{Nat} \rangle \equiv \langle \vdash \Pi(n : \text{Nat}). \text{Nat} \rangle$. Two Π -closures with convertible captured values and the same code are convertible by [Conv-cong]; the checker tries that first, as a fast path.
- [Conv-fun] Two function values F, G are convertible if their Π -types are, their captured values are pairwise convertible (none, for a top-level function), and at the generic call of their Π -type (Section A.1.4, item 10), with the same generic state Γ , arguments $\bar{a}$ and owned places $\bar{c}$, their observations agree: $\llbracket F(\bar{a}) \rrbracket_\Gamma^{\bar{c}} \equiv \llbracket G(\bar{a}) \rrbracket_\Gamma^{\bar{c}}$. For a codomain $\&T$ both observations use [Obs-borrow] with one shared fresh $\sigma_w : T$, so $\lambda(x \ y : \&\text{Nat}). x$ and $\lambda(x \ y : \&\text{Nat}). y$ differ: one leaves (σ_w, σ_y) in the owned places, the other (σ_x, σ_w) . So a function value’s normal form is its captured values together with the observation of its generic call, and two functions with different effects are never convertible.

There is no η rule and no η for `Unit` (an abstract $\sigma : \text{Unit}$ is not `()`). Proof irrelevance needs no rule: every proof value is $\star$. The relation is *least*: a function value that occurs in its own generic observation, as the head of a sealed program when its body is stuck at its generic call, is compared there by [Conv-refl] and [Conv-cong]. Read coinductively, [Conv-fun] would identify any two closures whose bodies are stuck at the generic call, such as $\lambda(x : \&\text{Nat}). \text{match } *x \{ Z \Rightarrow (), S _ \Rightarrow *x := Z \}$ and $\lambda(x : \&\text{Nat}). \text{match } *x \{ Z \Rightarrow *x := S Z, S _ \Rightarrow () \}$, and `J` along that identification proves `Eq Nat (S Z) Z`.

checker: `conv`, `convT`, `convFn`, `convPi`, `mkEqM`, `unitTop` (`Basic.lean`)

A.4 Typing

The typing judgement $\Omega \vdash t \Downarrow v : A \dashv \Omega'$ says that from Ω the term t runs to the value v , of type A , leaving Ω' ; its only other outcome is `err`, a type error. It is the machine with types: each rule below performs the step of the machine rule of the same name and also computes a type. Where the machine would be stuck on the checked program, typing splits instead ([Split]), so the typing judgement is never stuck. Callee bodies run in the machine, where stuck calls close off ([App-close]). We write $\Omega \vdash A \Downarrow T \text{ type}$ for $\Omega \vdash A \Downarrow T : s \dashv \Omega'$ with s a sort and Ω' discarded (a type position).

Sorts. Sorts are ordered $\text{Prop} < \text{Type}_0 < \text{Type}_1 < \dots$, and $s \sqcup s'$ is the larger. Universes are *not cumulative*: a type has exactly one sort, and no rule converts between sorts. The sort of a type value is: the declared sort s

of D for an inductive type $D(\bar{v})$ (Type_0 for Nat and Unit , Prop for False , True and And); Type_0 for $\&\text{T}$ (with T a data type); Prop for Eq ; Type_0 for Prop and Type_{i+1} for Type_i ; for a Π -closure, Prop if its codomain at the generic call has sort Prop (impredicativity), and otherwise the largest sort of its parameter types and codomain there; $\Delta(\sigma)$ for an abstract σ whose type is a sort; and for a sealed program $[L; C]$ whose head call's function has a declared codomain that is literally a sort s , the sort s .

checker: `sortOf, sealedSort?, isPropV`

A.4.1 Erasure

An occurrence of a term is *erased* when:

1. it stands in a type position (Section A.1), or inside one; or
2. it is a type former (a sort, Π , an inductive type $D(\bar{a})$, $\times$, $\&$, Eq , Id), since every type is formed on a private copy; or
3. it is a call whose callee *returns types* or *returns proofs*; or
4. it is a *declared proof*, judged from syntax alone: a constructor of an inductive declared in Prop (among them `refl` and $\langle h, k \rangle$); a match with no arms in a proof position; a match on a proof whose inductive has several constructors (Section A.4.6 requires every arm to be a declared proof); a J whose motive is syntactically a function into Prop , or whose last argument is a declared proof; a call whose callee returns proofs; a variable whose binding is flagged as a proof (a parameter whose declared type has declared sort Prop , or a `let` whose right-hand side is a declared proof; a captured value is flagged as its binding was); a `fix` whose Π -type returns proofs; `let x = t; u, t; u` or a match run directly when its tail (u , or the arm taken) is a declared proof; a stuck block when every arm is; or the bound term of `let x : A = t; u` when A has declared sort Prop .

A function value, top-level or local, *returns types* if the codomain term of its Π -type is syntactically a sort, and *returns proofs* if that term's *declared sort* is Prop . A type term has declared sort Prop when it is Id , Eq , or an inductive type $D(\bar{a})$ with D declared in Prop (such as False , $\top$ or $P \wedge Q$); a Π into such a term; a variable declared $: \text{Prop}$; a call whose head's codomain term has declared sort Prop ; a `let`, `;` or `match` whose tail does; or an ascription at Prop . Nothing is normalised: $U(n)$ with $U : \Pi(n : \text{Nat}). \text{Type}_0$ has declared sort Type_0 , whatever $U(n)$ computes to, and $V(Z)$ with $V : \Pi(n : \text{Nat}). U(n)$ does not have declared sort Prop although it computes to $\top$. A stuck block is erased exactly when *each* of its arms, as checked by `[Split]`, is a declared proof; it is never erased by clause 3 applied to its codomain, nor by its type inferred from the arms, since both are computed. No clause consults the type the typing judgement computes: reading clause 4 by the computed type while calls are classed by syntax, or reading a variable as a proof because its value is $\star$, each admits a closed proof of false (notes 2 and 3). A block that is not erased is always safe: its sealed programs re-run the arms, which make their own decisions. Every clause reads syntax, a declaration or a sort, so the two paths of Conjecture 3 take the same decisions. Notes 1–3 of Section A.6 show what goes wrong otherwise.

$$\frac{t \text{ erased} \quad \Omega \vdash_0 t \Downarrow v : A \dashv \Omega' \text{ by a confined run}}{\Omega \vdash t \Downarrow v^\bullet : A \dashv \Omega} \quad \text{T-ERASE}$$

Here $v^\bullet = \star$ if t is a declared proof (clause 4) and $v^\bullet = v$ otherwise, $\vdash_0$ applies the rule for t 's own form at the root, and a run that is not confined (Section A.2, `[Erase-err]`) is a type error. So an erased term is typed like any other term, on a private copy of the environment, and leaves no trace. Confinement is redundant for a correctly classified term, since the private copy already discards its effects; it is a fail-safe: if the two evaluation paths ever disagreed about whether a term is erased, the path that erases a term with outside effects would reject it, instead of silently discarding effects that the other path keeps. A proof may still mutate its own locals, and may hand outer places to other erased calls, as `AddMZero`'s recursive call `AddMZero(&p)` does.

checker: `eval, fnClass, propDecl, typeClass, jErased, evalType, onCopy` (`Env.lean`)

A.4.2 Places, sequencing and data

$$\begin{array}{c}
\frac{\langle \Omega, p \rangle \Downarrow \langle \Omega', v \rangle}{\Omega \vdash p \Downarrow v : \text{type}_\Omega(p) \dashv \Omega'} \text{ T-READ} \quad \frac{\langle \Omega, \&p \rangle \Downarrow \langle \Omega', v \rangle \quad \text{type}_\Omega(p) = T \text{ a data type}}{\Omega \vdash \&p \Downarrow v : \&T \dashv \Omega'} \text{ T-BORROW} \\
\\
\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad A \equiv \text{type}_{\Omega_1}(p) \quad \text{acc}_p^R(\Omega_1 \cdot v) = \Omega_2 \cdot v' \quad \text{drop}(\Omega_2, \text{content}_{\Omega_2}(p)) = \Omega_3}{\Omega \vdash p := t \Downarrow () : \text{Unit} \dashv \Omega_3[p \mapsto v']} \text{ T-ASSIGN} \\
\\
\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad \Omega_1 + (x : A \mapsto v) \vdash u \Downarrow w : B \dashv \Omega_2 + (x : A' \mapsto v') \quad \text{drop}(\Omega_2 \cdot w, v') = \Omega_3 \cdot w'}{\Omega \vdash \text{let } x = t; u \Downarrow w' : B \dashv \Omega_3} \text{ T-LET} \\
\\
\frac{\Omega \vdash A \Downarrow T \text{ type} \quad \Omega \vdash^T t \Downarrow v : T' \dashv \Omega_1 \quad T' \equiv T \quad \Omega_1 + (x : T \mapsto v) \vdash u \Downarrow w : B \dashv \Omega_2 + (x : T'' \mapsto v') \quad \text{drop}(\Omega_2 \cdot w, v') = \Omega_3 \cdot w'}{\Omega \vdash \text{let } x : A = t; u \Downarrow w' : B \dashv \Omega_3} \text{ T-LET-ANN} \\
\\
\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad \text{drop}(\Omega_1, v) = \Omega_2 \quad \Omega_2 \vdash u \Downarrow w : B \dashv \Omega_3}{\Omega \vdash t; u \Downarrow w : B \dashv \Omega_3} \text{ T-SEQ}
\end{array}$$

A binding stores the type of its value, and [Split] refines stored types, so A' in [T-Let] is A as refined. In [T-Let-ann], $\vdash^T$ checks $\mathfrak{t}$ against T : if $\mathfrak{t}$ is a match, it is [Split] with annotation T .

checker: placeType, evalCore (.place, .borrow, .assign, .letIn, .seq, .ascribe)

$$\frac{\begin{array}{c} C(g_1 : A_1, \dots, g_k : A_k) \text{ a constructor of } D(\bar{x} : \bar{X}) \\ \Omega \vdash \bar{t} \Downarrow^* \bar{v} : \bar{A}' \dashv \Omega' \quad A'_j \equiv A_j[\bar{a}/\bar{x}] \end{array}}{\Omega \vdash C(\bar{a}; \bar{t}) \Downarrow C(\bar{a}; \bar{v}) : D(\bar{a}) \dashv \Omega'} \text{ T-CTOR}$$

The typed argument judgement $\Downarrow^*$ is that of [T-Call]. In [T-Ctor] the parameters $\bar{a}$ are evaluated and typed against $\bar{X}$ as in [T-Ind] (premises omitted); where the examples omit them, they are inferred from the fields' types or the annotation. A constructor of an inductive declared in **Prop** is a declared proof, so [T-Erase] gives it the value $\star : \text{refl} : \tau$, and $\langle h, k \rangle : P \wedge Q$ for $h : P$ and $k : Q$.

checker: evalCore (.ctor, .zero, .succ, .tt), evalCtor

A.4.3 Type formers and transport

All type formers are erased, so each of these rules runs on a private copy ([T-Erase]).

$$\begin{array}{c}
\frac{}{\Omega \vdash \text{Prop} \Downarrow \text{Prop} : \text{Type}_0 \dashv \Omega} \text{ T-SORT} \quad \frac{}{\Omega \vdash \text{Type}_i \Downarrow \text{Type}_i : \text{Type}_{i+1} \dashv \Omega} \text{ T-TYPE} \\
\\
\frac{\text{inductive } D(\bar{x} : \bar{X}) : s := \dots \quad \Omega \vdash \bar{a} \Downarrow^* \bar{v} : \bar{A}' \dashv \Omega' \quad \Omega; (x_1 : T_1 \mapsto v_1, \dots, x_{i-1} : T_{i-1} \mapsto v_{i-1}) \vdash X_i \Downarrow T_i \text{ type} \quad A'_i \equiv T_i}{\Omega \vdash D(\bar{a}) \Downarrow D(\bar{v}) : s \dashv \Omega'} \text{ T-IND} \\
\\
\frac{\Omega \vdash A \Downarrow T \text{ type} \quad T \text{ a data type}}{\Omega \vdash \&A \Downarrow \&T : \text{Type}_0} \text{ T-REF} \quad \frac{\Omega \vdash A \Downarrow T \text{ type} \quad \Omega \vdash a \Downarrow v : T_a \equiv T \quad \Omega \vdash b \Downarrow w : T_b \equiv T}{\Omega \vdash \text{Eq } A a b \Downarrow \text{eq}(T, v, w) : \text{Prop}} \text{ T-EQ} \\
\\
\frac{\langle \Omega, \Pi(\bar{x} : \bar{A}).B \rangle \Downarrow \langle \Omega', \Pi \rangle \quad \text{sort}(\Pi) = s \quad B = \&T \text{ implies some } A_i = \&T_i}{\Omega \vdash \Pi(\bar{x} : \bar{A}).B \Downarrow \Pi : s \dashv \Omega'} \text{ T-PI}
\end{array}$$

A Π -type whose codomain is a borrow type must have a borrow parameter: a returned borrow always derives from a borrow argument, as in Rust's lifetime elision, except that `Ochr` has no `'static` borrows (note 7 of Section A.6); [Def] imposes the same condition on the type of every `fix`. [T-Ref] admits borrows of data types only (note 11). In [T-Ind] the parameters are typed as the arguments of a call are ([Call-type]). In [T-Eq] the sides are typed one after the other on the same private copy (the whole `Eq` is a type); in [T-Pi] the sort is computed at the generic arguments. The rule for `Id` is [Id] of Section A.3.

checker: `evalCore (.sort, .nat, .unit, .prod, .ref, .eq, .pi, .id, .tind), evalTInd, isDataType, sortOf`

$$\frac{\Omega \vdash A \Downarrow T \text{ type} \quad \Omega \vdash a \Downarrow v_a : T_a \equiv T \quad \Omega \vdash b \Downarrow v_b : T_b \equiv T \quad \Omega \vdash P \Downarrow F : \Pi \quad \Omega \vdash h \Downarrow \star : H \equiv \text{eq}(T, v_a, v_b) \quad \Omega \vdash F(v_a) \Downarrow P_a : s \quad \Omega \vdash F(v_b) \Downarrow P_b : s \quad \Omega \vdash t \Downarrow v : T_t \dashv \Omega' \quad T_t \equiv P_a \quad \text{T-J}}{\Omega \vdash J(A, a, b, P, h, t) \Downarrow v : P_b \dashv \Omega'}$$

`refl` proves τ ([T-Ctor]), and so by conversion every reflexive equation. `J` takes its endpoints explicitly, because `Eq A a a` computes to τ and no longer records them. Its motive may have any sort; with a motive into `Prop`, `J` is a proof and is erased. `A`, `a`, `b` and `P` are in type positions and `h` is a proof, so all five are typed on private copies, and the motive's calls $F(v_a)$, $F(v_b)$ are erased calls.

checker: `evalCore (.prim "J"), evalCtor, unifyParams`

A.4.4 Functions and calls

$$\frac{f \in \Sigma}{\Omega \vdash f \Downarrow f : \Sigma(f).type \dashv \Omega} \text{ T-GLOBAL}$$

$$\frac{c \in \Sigma}{\Omega \vdash c \Downarrow \Sigma(c).value : \Sigma(c).type \dashv \Omega} \text{ T-CONST} \quad \frac{\langle \Omega, \mathbf{fix} \dots \rangle \Downarrow \langle \Omega', F \rangle \quad \vdash F \text{ ok}}{\Omega \vdash \mathbf{fix} \dots \Downarrow F : \text{typeof}(F) \dashv \Omega'} \text{ T-FIX}$$

A `fix` is checked where it is formed, by [Def] at its own generic call, with its captured values; its value is $\star$ if its Π -type returns proofs.

checker: `evalCore (.const, .fix), checkFix`

[Call-type]. At the call point Ω of a call with function type $\Pi = \langle \bar{\kappa} \vdash \Pi(x_1 : A_1 \dots x_n : A_n).B \rangle$ and arguments $\bar{w}$ of types $\bar{A}'$, push the frame of captured values and bind the parameters one by one, on a private copy:

$$\frac{\Omega; (\bar{\kappa}, x_1 : T_1 \mapsto w_1, \dots, x_{i-1} : T_{i-1} \mapsto w_{i-1}) \vdash A_i \Downarrow T_i \text{ type} \quad A'_i \equiv T_i \quad (1 \leq i \leq n) \quad \Omega; (\bar{\kappa}, \bar{x} : \bar{T} \mapsto \bar{w}) \vdash B \Downarrow T_B \text{ type}}{\text{callty}_\Omega(\Pi, \bar{w} : \bar{A}') = T_B} \text{ CALL-TYPE}$$

The free variables of `B` other than $\bar{x}$ were captured when Π was formed; the caller's frames below are not in scope, but the footprints and owners of observations in `B` see them, which is how an induction hypothesis sees its caller's context. A borrow argument is moved into its parameter.

checker: `callType`

Typed arguments are evaluated like [Args], each into a temporary, collecting their types: $\Omega \vdash \bar{u} \Downarrow^* \bar{w} : \bar{A}' \dashv \Omega'$.

$$\frac{\Omega \vdash t_0 \Downarrow F : \Pi \dashv \Omega_0 \quad \Omega_0 \cdot F \vdash \bar{u} \Downarrow^* \bar{w} : \bar{A}' \dashv \Omega_1 \cdot F' \quad \text{no } w_i \text{ is } \perp \quad \text{callty}_{\Omega_1}(\Pi, \bar{w} : \bar{A}') = B' \quad \text{rec}_{\Omega_1}(F', \bar{w}) \quad \langle \Omega_1, F'(\bar{w}) \rangle \Downarrow_{\text{app}} \langle \Omega', v \rangle}{\Omega \vdash t_0(\bar{u}) \Downarrow v : B' \dashv \Omega'} \text{ T-CALL}$$

$$\frac{\Omega \vdash t_0 \Downarrow F : \Pi \dashv \Omega_0 \quad \Omega_0 \cdot F \vdash \bar{u} \Downarrow^* \bar{w} : \bar{A}' \dashv \Omega_1 \cdot F' \quad \text{no } w_i \text{ is } \perp \quad \text{callty}_{\Omega_1}(\Pi, \bar{w} : \bar{A}') = B' \quad F' \text{ returns proofs} \quad \text{rec}_{\Omega_1}(F', \bar{w})}{\Omega \vdash t_0(\bar{u}) \Downarrow \star : B' \dashv \Omega_1} \text{ T-CALL-PROOF}$$

[T-Call-proof] applies to a call whose callee returns proofs (Section A.4.1), and [T-Erase] then discards Ω_1 : a proof call's arguments are evaluated and checked, and its body is not run. In [T-Call] the callee's body runs in the machine ($\Downarrow_{\text{app}}$), since the callee was checked by its own [Def]; a call with a neutral head closes off at once.

checker: `evalCall`, `callFn`, `callType`

[Rec]. The [Rec] stack holds an entry (F, j, σ_j) for every enclosing function being checked that declares `by` x_j , where σ_j is the entry value of x_j at its generic call (through the borrow, for $\&\mathbf{D}$). $\text{rec}_{\Omega}(F, \bar{w})$ holds when, for every entry (F, j, σ_j) of the stack for this $\mathbf{F}$, the argument w_j , or its content if $w_j = \text{borrow}_{\ell} u$, is in $\text{sub}(\rho^*(\sigma_j))$: a strict subterm, through any field, of the entry value as refined so far. Entries stay on the stack while nested functions and block arms are checked, so recursive calls there are checked too.

checker: `recCheck`, `headOnly`

A.4.5 Matches and case splitting

Let $m = \text{match } p\{C_1 \Rightarrow t_1, \dots, C_m \Rightarrow t_m\}$ match on a place of inductive type $\mathbf{D}(\bar{a})$, with $\mathbf{D}$ declared in `Typeo` and constructors $C_1, \dots, C_m$ (the place's stored type must be $\mathbf{D}(\bar{a})$ for the $\mathbf{D}$ whose constructors the arms name: a match is never typed from its arms), and let $r_i = C_i(\sigma_{i1}, \dots, \sigma_{ik_i})$ be the refinement to C_i with fresh field values (Section A.1.4, item 7). On a constructor the typing judgement takes the arm, as the machine does, and checks only that arm. On an abstract value it splits ([Split]): each arm is checked under its refinement, and the match is then closed off as a stuck block, from which the rest of the program is checked once.

$$\begin{array}{c}
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = C_i(\bar{w}) \quad \Omega_1 \vdash t_i \Downarrow v : A \dashv \Omega'}{\Omega \vdash m \Downarrow v : A \dashv \Omega'} \quad \text{T-MATCH} \\
\\
\frac{\begin{array}{l} \text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = \sigma \quad \Omega_1[\sigma := r_i] \vdash t_i \Downarrow v_i : A_i \dashv \Omega_i \quad (1 \leq i \leq m) \\ B = T \text{ with } A_i \equiv T[r_i/\sigma] \text{ for all } i, \quad \text{or, without annotation, } B = A_1 \equiv \dots \equiv A_m \\ M = \{x \mid \Omega_1(x) \neq \perp, \text{ some } \Omega_i(x) = \perp\} \quad \langle \Omega_1, \text{block}(\Omega_1, m, B, M) \rangle \Downarrow \langle \Omega', v \rangle \end{array}}{\Omega \vdash m \Downarrow v : B \dashv \Omega'} \quad \text{SPLIT} \\
\\
\frac{\begin{array}{l} \text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = n \text{ a sealed program or an inert loan} \\ \sigma \text{ fresh, } \Delta(\sigma) = \text{type}_{\Omega_1}(p) \quad \Omega_1[\sigma/n]^+ \text{ with } n := \sigma \text{ recorded in } \rho \quad \vdash m \Downarrow v : B \dashv \Omega' \end{array}}{\Omega \vdash m \Downarrow v : B \dashv \Omega'} \quad \text{SPLIT-GEN}
\end{array}$$

In [Split] every arm is checked from Ω_1 , and the arms' states and values are discarded; each refinement is applied to the environment, the goal and every stored type. $\mathbf{T}$ is the annotation of an enclosing `let  $x : \mathbf{T} = m$` (checking mode, $\vdash^T$), whose type is then $\mathbf{T}$; without one, the arms' types must be convertible. The block is run by the machine, from the unrefined Ω_1 . [Split-gen] generalises a neutral head first (Lean's `generalize`): it replaces the neutral everywhere in Ω , the goal, the stored types and the annotation, and records it in ρ , so that the replacement persists when the neutral is derived again; the record is global, and survives a private copy in which it was made. Types are terms, so a match inside a type is checked in the same way.

checker: `evalMatch`, `evalMatchInd`, `splitArmsThenClose`, `ctorRefinement`, `generalizeNeutral`, `refine`

Tail position. The body of a definition is checked by a judgement $\Omega \vdash t \rightsquigarrow \mathcal{L}$ whose result is the finite set $\mathcal{L}$ of the *leaves* of its case tree: triples (Ω', v, A) , each in the state of its own path, with the goal refined along that path. A match is in tail position when only trailing drops follow it.

$$\begin{array}{c}
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = \sigma \quad \Omega_1[\sigma := r_i] \vdash t_i \rightsquigarrow \mathcal{L}_i \quad (1 \leq i \leq m)}{\Omega \vdash m \rightsquigarrow \mathcal{L}_1 \cup \dots \cup \mathcal{L}_m} \quad \text{TAIL-SPLIT} \\
\\
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = n \quad \Omega_1[\sigma/n]^+ \vdash m \rightsquigarrow \mathcal{L}}{\Omega \vdash m \rightsquigarrow \mathcal{L}} \quad \text{TAIL-GEN}
\end{array}$$

$$\begin{array}{c}
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = C_i(\bar{w}) \quad \Omega_1 \vdash t_i \rightsquigarrow \mathcal{L}}{\Omega \vdash m \rightsquigarrow \mathcal{L}} \text{TAIL-MATCH} \\
\\
\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad \Omega_1 + (x : A \mapsto v) \vdash u \rightsquigarrow \mathcal{L}}{\Omega \vdash \text{let } x = t; u \rightsquigarrow \{\text{pop}_x(\ell) \mid \ell \in \mathcal{L}\}} \text{TAIL-LET} \\
\\
\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad \text{drop}(\Omega_1, v) = \Omega_2 \quad \Omega_2 \vdash u \rightsquigarrow \mathcal{L}}{\Omega \vdash t; u \rightsquigarrow \mathcal{L}} \text{TAIL-SEQ} \\
\\
\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega' \quad t \text{ is not a match, let or sequence}}{\Omega \vdash t \rightsquigarrow \{(\Omega', v, A)\}} \text{TAIL-END}
\end{array}$$

Here $\text{pop}_x(\Omega_2 + (x : A' \mapsto v'), w, B) = (\Omega_3, w', B)$ with $\text{drop}(\Omega_2 \cdot w, v') = \Omega_3 \cdot w'$, and an annotated **let** checks t against its annotation as in [T-Let-ann]. In [Tail-gen], σ is fresh and $n := \sigma$ is recorded in ρ , as in [Split-gen]. An error on any path is an error.

checker: `checkTail`

A.4.6 Matches on proofs

A match whose arms name the constructors of an inductive declared in **Prop** is a match on a proof. Its scrutinee's value is $\star$ (a proof parameter is bound to $\star$ at the generic call, and an erased term returns $\star$), so it is decided by the scrutinee's type, $\text{type}_{\Omega_1}(p) = D(\bar{v})$, and never by its content, and nothing is refined. D is a *subsingleton* if it has no constructors, or one constructor all of whose field types have declared sort **Prop**; **False**, **True** and **And** are subsingletons. Arm i is checked from Ω_1^i , which is Ω_1 if every field of C_i is a proof, so that its fields hold $\star$ (Section A.1.4, item 1), and otherwise $\Omega_1[p \mapsto C_i(\bar{v}; \bar{w})]$ with $w_j = \star$ for a proof field and w_j a fresh abstract value of the field's type for a data field, as [Def] binds parameters. The second case arises only in an erased match, on its private copy.

$$\begin{array}{c}
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{type}_{\Omega_1}(p) = D(\bar{v}), D \text{ declared in Prop with the one constructor } C_1 \\
\Omega_1^1 \vdash t_1 \Downarrow v : A \dashv \Omega' \quad t_1 \text{ is a declared proof unless } D \text{ is a subsingleton}}{\Omega \vdash \text{match } p\{C_1 \Rightarrow t_1\} \Downarrow v : A \dashv \Omega'} \text{T-MATCH-PROP} \\
\\
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{type}_{\Omega_1}(p) = D(\bar{v}), D \text{ declared in Prop with constructors } C_1, \dots, C_m, m \neq 1 \\
\Omega_1^i \vdash t_i \Downarrow \star : A_i \dashv \Omega_i, t_i \text{ a declared proof } (1 \leq i \leq m) \\
B = T \text{ with } A_i \equiv T, \text{ or, for } m \geq 2 \text{ without annotation, } B = A_1 \equiv \dots \equiv A_m}{\Omega \vdash m \Downarrow \star : B \dashv \Omega_1} \text{T-MATCH-ERASED} \\
\\
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{type}_{\Omega_1}(p) = D(\bar{v}), D \text{ declared in Prop} \quad \Omega_1^i \vdash t_i \rightsquigarrow \mathcal{L}_i \quad (1 \leq i \leq m)}{\Omega \vdash m \rightsquigarrow \mathcal{L}_1 \cup \dots \cup \mathcal{L}_m} \text{TAIL-PROP}
\end{array}$$

In [T-Match-erased], T is the annotation of an enclosing **let** $x : T = m$; with no arms, the annotation, or tail position, is the only source of the result type. [T-Match-erased] is a declared proof (Section A.4.1), so [T-Erase] applies to it. In [Tail-prop], each t_i is a declared proof unless D is a subsingleton; with no arms $\mathcal{L} = \emptyset$, and [Def]'s check on every leaf holds vacuously, which is ex falso. *Subsingleton elimination* is the side condition “a declared proof unless D is a subsingleton”: it is Lean's, and without it proof irrelevance is inconsistent (note 10 of Section A.6). The decision is stable under refinement, which changes a type's parameters but never its head; a scrutinee whose type is not (yet) an inductive, such as $[\text{Le}(\sigma, \sigma')]$ before a split, has no rule, so it is a type error until a split has computed its type. The machine takes the same decisions from syntax alone: [Match-prop] runs a one-constructor match, and every other match on a proof, and every match with no arms in a proof position, is a declared proof it never runs; a match with no arms elsewhere is stuck.

checker: `byTypeMatch`, `scrutType`, `evalMatchByType`, `checkTail`, `bindDataFields`, `largeElim`, `fieldTermIsProp`

A.4.7 Definitions

[Def]. Let F be a top-level function or a closure with captured values $\bar{\kappa}$ and code $\mathbf{fix} \ f(x_1 : A_1 \dots x_n : A_n) : B \ [\mathbf{by} \ x_j] := b$, and let $\Pi = \text{typeof}(F)$, with generic state $\Gamma = \Gamma(\Pi)$, parameter frame $\varphi = \varphi(\Pi)$ and parameter types $\bar{T}$ (Section A.1.4, item 10). Let φ^+ be φ with $f : \Pi \mapsto F$ added when F declares $\mathbf{by}$.

$$\begin{array}{c}
 f \text{ occurs in } b \text{ only as the head of a call } [\mathbf{by} \ x_j] : T_j \in \{D(\bar{v}), \&D(\bar{v})\}, D \text{ declared in } \text{Type}_0 \\
 B = \&T \text{ implies some } T_i = \&T'_i \\
 \Gamma; \varphi \vdash B \Downarrow G \text{ type} \quad \Gamma; \varphi^+ \vdash b \rightsquigarrow \mathcal{L} \quad \text{with goal } G \text{ and the } [\text{Rec}] \text{ entry } (F, j, \sigma_j) \text{ pushed} \quad \text{DEF} \\
 \text{for every } (\Omega', v, A) \in \mathcal{L} \text{ with refined goal } G' : \text{pop}(\Omega', v) \text{ succeeds and } A \equiv G' \\
 \hline
 \vdash F \text{ ok}
 \end{array}$$

In the first premise, f must also occur only as a call head inside nested functions and block arms. The goal G is the [Call-type] of the generic call $F(\bar{a}) : B$ evaluated with each parameter bound to its generic argument, borrow parameters to borrows of the fresh owned places c_i , proof parameters to $\star$. Parameter types are evaluated left to right and stored with their bindings, where [Split] refines them. Popping the body's frame drops its bindings, so each borrow parameter ends and its owned place receives the final content; the result's type must then convert to the goal as refined on that path. A top-level f is added to Σ , with its type, code and erasure flag, before its body is checked, so that its recursive calls resolve; it is kept only if the check succeeds.

checker: `checkFix, checkDef` (Check.lean)

$$\begin{array}{c}
 D \notin \Sigma \quad s \in \{\text{Type}_0, \text{Prop}\} \quad m \geq 0 \quad \varepsilon; (x_1 : T_1 \mapsto \sigma_1, \dots, x_{i-1} : T_{i-1} \mapsto \sigma_{i-1}) \vdash X_i \Downarrow T_i \text{ type} \quad (1 \leq i \leq l) \\
 \text{for every field } g_{ij} : A_{ij} : A_{ij} \text{ is a parameter, or is built from } D(\bar{x}), \text{ declared inductive types and } \times \\
 \Sigma + D; (\bar{x} : \bar{T} \mapsto \bar{\sigma}) \vdash A_{ij} \Downarrow T_{ij} : s_{ij} \quad s_{ij} \in \{\text{Prop}, \text{Type}_0\} \\
 \hline
 \vdash (\mathbf{inductive} \ D(\bar{x} : \bar{X}) : s := C_1(\bar{g}_1 : A_1) \mid \dots \mid C_m(\bar{g}_m : A_m)) \text{ ok} \\
 \varepsilon \vdash A \Downarrow T \text{ type} \quad \varepsilon \vdash t \Downarrow v : T' \dashv \Omega' \quad T' \equiv T \quad \Sigma(c) = (T, v^\bullet) \quad \text{CONST} \\
 \hline
 \vdash (c : A := t) \text{ ok}
 \end{array}$$

Parameters are checked like the parameters of a [Def], each bound to a fresh abstract value (or $\star$) while later ones and the fields are checked, and the recursive occurrences of D are at exactly those parameters (they are uniform: there are no indices). Field types are first-order data or parameters: they may mention the type being declared, but contain no Π and no $\&$, which is strict positivity in its simplest form (note 4 of Section A.6). There may be no constructors (**False**), and the sort may be **Prop**, for any fields; what a match on a proof may produce is restricted instead (Section A.4.6). A definition without parameters is a constant: its body is checked against its type from the empty environment, and its value is stored ($v^\bullet = \star$ if T has declared sort **Prop**). A definition may not mention itself in its type.

checker: `checkInd, checkDef` (Check.lean)

A.5 Well-formed environments

A state (Ω, Δ, Σ) is *well formed* when the following hold. They are the four conditions of Section 7.1, spelled out, plus typing.

1. **Borrows are unique and outermost.** Each label ℓ has at most one borrow_ℓ in Ω , and a borrow occurs only as the whole value of a binding or temporary: never inside data, a sealed program, a closure or a type.
2. **Loans are bound.** Every loan_ℓ in Ω has its borrow_ℓ in Ω , except inside a [Seal] run, where a loan whose borrow lies outside the run is inert. Outside sealed programs a label occurs as a loan at most once; inside sealed programs it may occur any number of times, since the hole of a returned borrow sits in the fill of every borrowed argument (Figure 12).
3. **Loans are acyclic.** The relation “ loan_ℓ occurs in the content of borrow_m ” is acyclic, so owners is well defined.
4. **Captures are values.** Closures and Π -closures capture no borrow and no $\perp$, and sealed programs are closed.
5. **Arguments are exclusive.** At every call point, each argument is loan-free or is $\text{borrow}_\ell u$ with u loan-free. This is [Close]'s precondition, and [Access] maintains it.

6. **Values have their types.** For each binding $x : A \mapsto v$: $v = \perp$ (moved out or ended), or $v = \text{loan}_\ell$ and the content of borrow_ℓ has type A , or v has type A (a neutral having the type recorded for it). Every σ occurring in Ω , in a stored type or in the goal has a type in Δ , and every top-level name has an entry in Σ .

That the machine preserves conditions 1 to 5 is property 2 of Figure 8, mechanised for the first-order fragment of Section 7.3.

A.6 Notes on the definition

Notes 1–11 explain the side conditions of Figure 7 that are finest-grained in this appendix: each gives a program that, without the condition, is a closed proof of a false equation (or, for note 8, makes normalisation diverge); by the disjointness rule of Section A.3, an equation between distinct constructors, such as Eq Nat Z (S Z) , is **False** itself. Notes 12–27 record choices where the definition could have gone either way.

1. **A function’s erasure class is read from its codomain term.** A local **fix** is formed, and so checked by [Def], each time the term containing it is evaluated, and its codomain may depend on captured values. Deciding its class from the codomain’s *value* would give different answers on the two paths:

```
U(n : Nat) : Type0 := match n { Z => Prop, S _ => Prop }
V(n : Nat) : U(n) := match n { Z =>  $\tau$ , S _ =>  $\tau$  }
LieL(n : Nat) : Id Nat (let h =  $\lambda(x : \&\text{Nat}) : U(n) \Rightarrow (*x := S\ Z; V(n)); \text{let } c = Z;$ 
  h(&c); c) (S Z) := refl
BoomL : Eq Nat Z (S Z) := LieL(Z)
```

At the generic n the codomain is $\lceil U(\sigma) \rceil$, not a sort, so $h(\&c)$ runs and the statement is τ . At $n = Z$ the codomain is $U(Z) = \text{Prop}$, a sort, so $h(\&c)$ would be erased, c would stay Z , and $\text{LieL}(Z) : \text{Eq Nat Z (S Z)}$. Read from the term, $U(n)$ is not syntactically a sort and has declared sort Type_0 , so $h(\&c)$ runs on both paths.

2. **A stuck block is erased only when every arm is a declared proof.** Treating the block as a call, whose codomain is the match’s type, erases a block whose type is a sort, while the same match run directly is not a call and is not erased:

```
LieB(n : Nat) : Id Nat (let c = Z; let  $\tau = \text{match } n \{ Z \Rightarrow (c := S\ Z; \tau), S\_ \Rightarrow (c := S\ Z; \tau) \}; c) Z := refl$ 
BoomB : Eq Nat (S Z) Z := LieB(Z)
```

Nor may the block be erased by its type inferred from the arms, which is computed. With $g : \Pi(y : \text{Nat}). V(Z)$ and $f := \lambda(x : \&\text{Nat}) : V(Z) \Rightarrow (*x := S\ Z; g(\theta))$, the block $\text{match } m \{ Z \Rightarrow f(\&c), S_ \Rightarrow f(\&c) \}$ has inferred type $V(Z)$, which computes to τ , of sort Prop ; but f returns data (the declared sort of $V(Z)$ is not Prop), so at $m = Z$ the match runs $f(\&c)$ and writes c , while the block erased at the generic call would not. A block that is not erased is always safe, because its sealed programs re-run the arms, which decide for themselves.

3. **A proof is declared, not computed.** Classifying a variable as a proof because its value is $\star$ differs between the paths:

```
LieH(g :  $\Pi(y : \text{Nat}). V(Z)$ ) : Id Nat (let c = Z; let h = g( $\theta$ ); (c := S Z; h); c) (S Z) := refl
BoomH : Eq Nat Z (S Z) := LieH(( $\lambda(y : \text{Nat}) : V(Z) \Rightarrow \text{refl}$ ))
```

At the generic call h holds a sealed program, so $(c := S\ Z; h)$ runs and the statement is τ ; at the instance $g(\theta)$ returns $\star$, a value-based reading erases the sequence, c stays Z , and the statement is Eq Nat Z (S Z) . Reading clause 4 of Section A.4.1 by the type the typing judgement computes would be stable on its own, but combined with the syntactic class of calls it is the block example of note 2; so every clause is read from declarations.

4. **Inductive declarations are strictly positive.** A negative field gives a closed proof of **False** that is never run:

```

inductive Bad : Type0 := Mk(f :  $\Pi(x : \text{Bad}). \text{False}$ )
L(b : Bad) : False := match b { Mk(f) => f(b) }
Bad4 : False := L(Mk( $\lambda(x : \text{Bad}) : \text{False} \Rightarrow L(x)$ ))

```

$L(\dots)$ is a proof call, so it is typed by [Call-type] alone and its diverging body never runs. [Ind]’s first-order fields exclude **Bad**.

5. **Generalisation records are global, and fresh names are never reused.** A match inside a type whose scrutinee is a sealed program is generalised on the type’s private copy, and the formed type mentions the new σ :

```

inductive Box := Mk(x : Nat)
Double(n : Nat) : Nat by n := match n { Z => Z, S p => S (S (Double(p))) }
Esc(n : Nat, m : Box) : Id Nat (let b = Double(n); match b { Z => 0, S _ => 1 }) (match
m { Mk(x) => match x { Z => 0, S _ => 1 } }) :=
  match m { Mk(x) => match x { Z => refl, S _ => refl } }
Bad5 : Eq Nat 1 0 := Esc(1, Mk(0))

```

If the record $[\text{Double}(\sigma)] := \sigma_g$ were discarded with the copy, and σ_g ’s name reissued to the field x when the body splits m , the goal would equate the two and **Esc** would check; its instance is **Eq Nat 1 0**.

6. **A borrow-typed result is observed through a written value** ([Obs-borrow]). Ending a returned borrow with its current content forgets where it points, so $\text{PickX}(x, y : \&\text{Nat}) : \&\text{Nat} := x$ and $\text{PickY}(x, y : \&\text{Nat}) : \&\text{Nat} := y$ would be convertible; transport from $\Pi(x y : \&\text{Nat}). \text{Id Unit (let } r = h(x, y); *r := S Z) (*x := S Z)$, true of PickX , then proves it of PickY , whose instance at two zeros is **Eq Nat 0 1 $\wedge$ Eq Nat 1 0**, which is **False**.
7. **A borrow-returning function type needs a borrow parameter** ([T-Pi], [Def]). Closing off a call $g(5)$ with $g : \Pi(n : \text{Nat}). \&\text{Nat}$ would return a borrow whose hole lies in no owner, so writes through it are unobservable. The function $\text{PF}(x : \&\text{Nat}, e : \text{Id Unit } (*x := 0) (*x := 1)) : \text{False} := e$ checks, since at its generic call the statement observes the owner of x and computes to **Eq Nat 0 1**, which is **False**. Then $\text{QF}(g : \Pi(n : \text{Nat}). \&\text{Nat}) : \text{False} := \text{PF}(g(5), \text{refl})$ checks too, because at this call site the statement observes nothing and computes to $\top$: it proves that $\Pi(n : \text{Nat}). \&\text{Nat}$ is empty, although safe Rust inhabits it with a leaked 'static borrow, and any such opaque inhabitant gives a closed proof of **False**.
8. **The head guard covers neutral-headed calls** ([App-neutral-head]). Closing off a sealed program’s head call when its head is an abstract function with codomain $\&\text{T}$ and two or more borrow arguments creates a new hole; reading an owner ends it, re-normalises a sealed program of the same shape, and so on for ever.
9. **Confinement is a fail-safe.** For a correctly classified term the private copy already discards its effects. If the two paths ever disagreed about whether a term is erased, the path that erases a term with outside effects rejects it rather than silently discarding effects the other path keeps. A place *outlives* an erased term when its root is a position of the environment the term starts from; the exception for erased calls covers exactly the borrows and moves that evaluate their arguments.
10. **Subsingleton elimination** (Section A.4.6). A match on a proof of an inductive with two constructors cannot produce data. With **inductive Bool := false | true**:

```

inductive Or (P : Prop) (Q : Prop) : Prop := Inl(p : P) | Inr(q : Q)
IsL(h : Or(True, True)) : Bool := match h { Inl(p) => true, Inr(q) => false }
Irr(h : Or(True, True), k : Or(True, True)) : Eq Bool (IsL(h)) (IsL(k)) := refl
Boom : False := Irr(Inl(refl), Inr(refl))

```

Without the condition, **IsL** and **Irr** check: at the generic call h and k are both $\star$, so both sides are the stuck $[\text{IsL}(\star)]$. **Boom** is still rejected, because $\text{Inl}(\text{refl})$ is a proof, whose value is $\star$, so **Irr**’s instance is **Eq Bool $[\text{IsL}(\star)] [\text{IsL}(\star)]$** , which is **True**. Only if a proof also kept its constructor would **Boom** prove **False**, as it would in a system where proofs keep their constructors. The condition is required all the same: **IsL** has no meaning in the set model once proofs are irrelevant (**Eq Bool (IsL(Inl(refl))) (IsL(Inr(refl)))**),

provable by `refl`, reads `Eq Bool true false` there); `IsL(Inl(refl))` is a closed `Bool` that is stuck for ever, against canonicity and adequacy; and a two-constructor match on a proof, which cannot take an arm, has a value only because every arm is a proof. For `False`, `True` and `And` the type determines the arm and the fields are proofs, so nothing is chosen.

11. **Borrows are only of data, and & only at the top of a declared type** (`[T-Ref]`, `[T-Borrow]`). If `&A` were allowed for every type `A`, a borrow of a type would quantify over `Type0` inside `Type0`:

```
Impred : Type0 :=  $\Pi(x : \&Type_0) (a : *x). *x$ 
PolyId(x : &Type0, a : *x) : *x := a
SelfApp(u : Unit) : Impred := let T = Impred; PolyId(&T, PolyId)
```

$\Pi(X : Type_0)(a : X)$. `X` lives in `Type1`, but through the borrow it would live in `Type0`, which becomes impredicative: with the impredicative `Prop : Type0` below it, the rules contain Girard's System U^- , in which Hurkens' paradox is a closed term, and no set model exists. And if a codomain could compute to a borrow type, as in `F(n : Nat, x : &Nat) : (match n { Z => &Nat, S _ => Nat })`, `[Close]` would read the data row at the generic call, where the match is stuck, while `F(0, &a)` returns a live borrow at the instance; `G(n, a) := let r = F(n, &a); let r2 = r; let r3 = r; a` is accepted at the generic call and reads a moved borrow at `n = 0`. Because data is read from the evaluated head of `A`, a type variable is rejected too: `SwapT(A : Type0, x : &A, y : &A)` is not well formed, since `A` could be `Type0` itself.

12. **Proofs are not run.** A proof call's arguments are evaluated and its `[Call-type]` and `[Rec]` checked, on a private copy; its body is never run (`[T-Call-proof]`), and the machine skips proofs altogether (`[Erase-proof]`). Running and skipping agree, since erased terms run on a private copy.
13. **The order of [Access].** Loans are ended from the root of the place outward, then left to right inside its content. The resolution should not depend on the order (property 7 of Figure 8, proved for two endings).
14. **A match on a constructor checks only the arm taken**, in typing as in the machine (`[T-Match]`).
15. **[Close]'s row is chosen by the declared codomain** (`Unit`, `&T`, anything else), as Conjecture 3 (4) requires. A codomain that computes to `Unit` without being syntactically `Unit` gets the data row, and there is no η for `Unit`, so a statement such as `Id Unit (let c = *x; G(&c, Z)) ()`, where `G`'s codomain `UU(n)` has `UU(Z) = Unit`, needs induction on `*x` rather than `refl`.
16. **A discarded value** (`t; u`) holding a live loan is an error, like a dying owned binding.
17. **Stuck-block moves.** A whole variable is moved into a block if some arm, when checked, leaves it $\perp$, or if it is a borrow variable read or assigned as a whole. Captures are ordered by the position of their root, then by length, and their arguments are evaluated in that order.
18. **Footprint.** A place rooted at a borrow variable contributes its owners even when it is only read, and a variable holding $\perp$ contributes nothing. Owners are positions, so a temporary can be an owner, when a type is formed while arguments are in flight.
19. **J.** `A`, `a`, `b` and `P` are type positions and `h` is a proof, so none of them runs in the machine; `J`'s value is `t`'s when `a  $\equiv$  b`, and it is stuck otherwise. Reducing regardless of the endpoints would be equality reflection for open terms: under a hypothesis equating `Nat  $\rightarrow$  Nat` with `(Nat  $\rightarrow$  Nat)  $\rightarrow$  Nat`, casts give an untyped λ -calculus inside the checker, and a cast of 5 to `Bool` puts a number where a boolean is expected. The motive may have any sort. `J` is a declared proof when its motive is syntactically a function into `Prop`, or when `t` is a declared proof.
20. **Universes.** `&A` lives in `Type0`, an inductive type in its declared sort (so `A  $\times$  B` in `Type0`, its parameters being in `Type0`), and Π is impredicative in `Prop`. `And`'s parameters have type `Prop`, so `[T-Ind]` requires both conjuncts to be propositions.
21. **Recursion** requires the declared decreasing parameter to have an inductive type declared in `Type0`, or to be a borrow of one (so recursion is never on a proof); strict subterms range over all fields.
22. **Conversion** is the least relation of Section A.3. Π -types are compared by instantiating their binders at generic arguments (`[Conv-pi]`); comparing their captured values and code instead would reject a closure that captures a variable where a closed Π -type is expected, and would not unfold type abbreviations under binders. There is no η rule, and no η for `Unit`.

23. **An unannotated non-tail match** needs its arms' types, each computed in its own refined state, to be convertible; an annotated one checks each arm against the annotation refined for that arm.
24. **Syntax.** `Nat` and `Unit` are presented as declared types. The implementation builds them in with the same behaviour, declares `Pair`, `False`, `True` and `And` in its library exactly as in Figure 1, and checks those declarations like any other (`[Ind]`); it also has conveniences outside the core (`cong`, `trans`, `symm`, the ascription $(t : A), \lambda$ and $\rightarrow$).
25. **A match on a proof is decided by the scrutinee's stored type** (Section A.4.6), the type kept with its binding and refined by `[Split]`, not by its declared type term. In `SubM` (Section 2), `h` is declared `Le(y, *x)`, and its stored type becomes `False` after the two splits, so `match h {}` checks there. Requiring the declared term itself to be an inductive would also be stable, and would make `SubM` write `let e : False = h; match e {}`. This consults a normal form, but only in a way that fails safe: a split only refines the stored type, which keeps an inductive head, so a less refined path sees a neutral and rejects, and never sees a different head. A match with no arms in a proof position is a declared proof. Anywhere else it is stuck when reached, which happens only while a sealed program is re-normalised under a refinement that makes a hypothesis false; it then stays a neutral instead of producing a value of the wrong type.
26. **Injectivity and disjointness.** `eq` takes two values built by the same constructor apart into the equations between their fields, and sends two distinct constructors to `False`, so `Eq` computes on constructor values by structure; pairs have no rule of their own. Both are sound in the model, where constructors are injective and have disjoint images; each is an instance of `propext`.
27. **Captured values keep their declared types.** A closure or Π -closure records, with each captured value, the declared type and proof flag of the binding it was read from. So a λ or Π whose body reads a captured proof, or captured neutral data such as a sealed program, can be typed: `let n = *x after AddM(&*x, 1)`, followed by `$\lambda(y : \text{Nat}) : \text{Nat} \Rightarrow n$` , is accepted. Whether a captured variable is a proof is read from its declaration, never from its value $\star$ (note 3).